projectmanager Application with Library

0.0.1

Generated by Doxygen 1.16.1

1 Directory Hierarchy 1
1.1 Directories . . . 1

2 Namespace Index 7
2.1 Package List . . . 7

3 Hierarchical Index 9
3.1 Class Hierarchy . . . 9

4 Class Index 11
4.1 Class List . . . 11

5 File Index 13
5.1 File List . . . 13

6 Directory Documentation 15
6.1 projectmanager-app/src/main/java/com/projectmanagement/app Directory Reference . . . 15
6.2 projectmanager-app/src/main/java/com Directory Reference . . . 15
6.3 projectmanager-app/src/main/java Directory Reference . . . 15
6.4 projectmanager-app/src/main/java/com/projectmanagement/lib Directory Reference . . . 15
6.5 projectmanager-app/src/main Directory Reference . . . 16
6.6 projectmanager-app/src/main/java/com/projectmanagement/lib/models Directory Reference . . . 16
6.7 projectmanager-app/src/main/java/com/projectmanagement Directory Reference . . . 16
6.8 projectmanager-app Directory Reference . . . 16
6.9 projectmanager-app/src/main/java/com/projectmanagement/lib/services Directory Reference . . . 16
6.10 projectmanager-app/src Directory Reference . . . 17
6.11 projectmanager-app/src/main/java/com/projectmanagement/lib/storage Directory Reference . . . 17

7 Namespace Documentation 19
7.1 Package com.projectmanagement.app . . . 19
7.2 Package com.projectmanagement.lib.models . . . 19
7.3 Package com.projectmanagement.lib.services . . . 19
7.4 Package com.projectmanagement.lib.storage . . . 20

8 Class Documentation 21
8.1 com.projectmanagement.lib.models.BaseEntity Class Reference . . . 21
8.1.1 Detailed Description . . . 21
8.1.2 Constructor & Destructor Documentation . . . 22
8.1.2.1 BaseEntity() [1/2] . . . 22
8.1.2.2 BaseEntity() [2/2] . . . 22
8.1.3 Member Function Documentation . . . 22
8.1.3.1 displayDetails() . . . 22
8.1.3.2 getCreatedDate() . . . 22
8.1.3.3 getId() . . . 23
8.1.3.4 setCreatedDate() . . . 23

8.1.3.5 setId() . . . 23
8.2 com.projectmanagement.lib.storage.BinaryProjectRepository Class Reference . . . 24
8.2.1 Detailed Description . . . 24
8.2.2 Member Function Documentation . . . 24
8.2.2.1 delete() . . . 24
8.2.2.2 findAll() . . . 25
8.2.2.3 findById() . . . 25
8.2.2.4 save() . . . 26
8.2.2.5 update() . . . 26
8.3 com.projectmanagement.lib.storage.BinaryTaskRepository Class Reference . . . 26
8.3.1 Detailed Description . . . 27
8.3.2 Member Function Documentation . . . 27
8.3.2.1 delete() . . . 27
8.3.2.2 findAll() . . . 28
8.3.2.3 findById() . . . 28
8.3.2.4 save() . . . 28
8.3.2.5 update() . . . 29
8.4 com.projectmanagement.lib.storage.BinaryUserRepository Class Reference . . . 29
8.4.1 Detailed Description . . . 30
8.4.2 Member Function Documentation . . . 30
8.4.2.1 delete() . . . 30
8.4.2.2 findAll() . . . 30
8.4.2.3 findById() . . . 31
8.4.2.4 save() . . . 31
8.4.2.5 update() . . . 31
8.5 com.projectmanagement.lib.storage.IRepository< T > Interface Template Reference . . . 32
8.5.1 Detailed Description . . . 32
8.5.2 Member Function Documentation . . . 33
8.5.2.1 delete() . . . 33
8.5.2.2 findAll() . . . 33
8.5.2.3 findById() . . . 33
8.5.2.4 save() . . . 34
8.5.2.5 update() . . . 34
8.6 com.projectmanagement.app.Main Class Reference . . . 34
8.6.1 Detailed Description . . . 34
8.6.2 Member Function Documentation . . . 35
8.6.2.1 main() . . . 35
8.7 com.projectmanagement.lib.storage.MySQLProjectRepository Class Reference . . . 35
8.7.1 Detailed Description . . . 36
8.7.2 Constructor & Destructor Documentation . . . 36
8.7.2.1 MySQLProjectRepository() . . . 36
8.7.3 Member Function Documentation . . . 36

8.7.3.1 delete() . . . 36
8.7.3.2 findAll() . . . 37
8.7.3.3 findById() . . . 37
8.7.3.4 getConnection() . . . 38
8.7.3.5 save() . . . 38
8.7.3.6 update() . . . 38
8.8 com.projectmanagement.lib.storage.MySQLTaskRepository Class Reference . . . 39
8.8.1 Detailed Description . . . 40
8.8.2 Constructor & Destructor Documentation . . . 40
8.8.2.1 MySQLTaskRepository() . . . 40
8.8.3 Member Function Documentation . . . 40
8.8.3.1 delete() . . . 40
8.8.3.2 findAll() . . . 41
8.8.3.3 findById() . . . 41
8.8.3.4 getConnection() . . . 42
8.8.3.5 save() . . . 42
8.8.3.6 update() . . . 43
8.9 com.projectmanagement.lib.storage.MySQLUserRepository Class Reference . . . 43
8.9.1 Detailed Description . . . 44
8.9.2 Constructor & Destructor Documentation . . . 44
8.9.2.1 MySQLUserRepository() . . . 44
8.9.3 Member Function Documentation . . . 44
8.9.3.1 delete() . . . 44
8.9.3.2 findAll() . . . 45
8.9.3.3 findById() . . . 45
8.9.3.4 getConnection() . . . 46
8.9.3.5 save() . . . 46
8.9.3.6 update() . . . 46
8.10 com.projectmanagement.lib.models.Project Class Reference . . . 47
8.10.1 Detailed Description . . . 48
8.10.2 Constructor & Destructor Documentation . . . 48
8.10.2.1 Project() [1/2] . . . 48
8.10.2.2 Project() [2/2] . . . 48
8.10.3 Member Function Documentation . . . 49
8.10.3.1 addTask() . . . 49
8.10.3.2 displayDetails() . . . 49
8.10.3.3 getDescription() . . . 49
8.10.3.4 getName() . . . 50
8.10.3.5 getTasks() . . . 50
8.10.3.6 removeTask() . . . 50
8.10.3.7 setDescription() . . . 51
8.10.3.8 setName() . . . 51

8.10.3.9 setTasks() . . . 51
8.11 com.projectmanagement.lib.services.ProjectService Class Reference . . . 52
8.11.1 Detailed Description . . . 52
8.11.2 Constructor & Destructor Documentation . . . 52
8.11.2.1 ProjectService() . . . 52
8.11.3 Member Function Documentation . . . 52
8.11.3.1 addTaskToProject() . . . 52
8.11.3.2 createProject() . . . 53
8.11.3.3 getAllProjects() . . . 54
8.12 com.projectmanagement.lib.services.ReportService Class Reference . . . 54
8.12.1 Detailed Description . . . 54
8.12.2 Constructor & Destructor Documentation . . . 54
8.12.2.1 ReportService() . . . 54
8.12.3 Member Function Documentation . . . 55
8.12.3.1 generateGeneralSystemReport() . . . 55
8.12.3.2 generateTaskStatusReport() . . . 55
8.13 com.projectmanagement.lib.storage.RepositoryFactory Class Reference . . . 56
8.13.1 Detailed Description . . . 56
8.13.2 Constructor & Destructor Documentation . . . 56
8.13.2.1 RepositoryFactory() . . . 56
8.13.3 Member Function Documentation . . . 56
8.13.3.1 getProjectRepository() . . . 56
8.13.3.2 getTaskRepository() . . . 57
8.13.3.3 getUserRepository() . . . 57
8.14 com.projectmanagement.lib.storage.SQLiteProjectRepository Class Reference . . . 58
8.14.1 Detailed Description . . . 58
8.14.2 Constructor & Destructor Documentation . . . 58
8.14.2.1 SQLiteProjectRepository() . . . 58
8.14.3 Member Function Documentation . . . 59
8.14.3.1 delete() . . . 59
8.14.3.2 findAll() . . . 59
8.14.3.3 findById() . . . 60
8.14.3.4 getConnection() . . . 60
8.14.3.5 save() . . . 60
8.14.3.6 update() . . . 61
8.15 com.projectmanagement.lib.storage.SQLiteTaskRepository Class Reference . . . 62
8.15.1 Detailed Description . . . 62
8.15.2 Constructor & Destructor Documentation . . . 62
8.15.2.1 SQLiteTaskRepository() . . . 62
8.15.3 Member Function Documentation . . . 63
8.15.3.1 delete() . . . 63
8.15.3.2 findAll() . . . 63

8.15.3.3 findById() . . . 64
8.15.3.4 getConnection() . . . 64
8.15.3.5 save() . . . 65
8.15.3.6 update() . . . 65
8.16 com.projectmanagement.lib.storage.SQLiteUserRepository Class Reference . . . 66
8.16.1 Detailed Description . . . 66
8.16.2 Constructor & Destructor Documentation . . . 66
8.16.2.1 SQLiteUserRepository() . . . 66
8.16.3 Member Function Documentation . . . 67
8.16.3.1 delete() . . . 67
8.16.3.2 findAll() . . . 67
8.16.3.3 findById() . . . 68
8.16.3.4 getConnection() . . . 68
8.16.3.5 save() . . . 69
8.16.3.6 update() . . . 69
8.17 com.projectmanagement.lib.storage.StorageConfig Class Reference . . . 70
8.17.1 Detailed Description . . . 70
8.17.2 Constructor & Destructor Documentation . . . 70
8.17.2.1 StorageConfig() . . . 70
8.17.3 Member Function Documentation . . . 70
8.17.3.1 getActiveStorageType() . . . 70
8.17.3.2 setActiveStorageType() . . . 71
8.18 com.projectmanagement.lib.storage.StorageType Enum Reference . . . 71
8.18.1 Detailed Description . . . 71
8.18.2 Member Data Documentation . . . 71
8.18.2.1 BINARY_FILE . . . 71
8.18.2.2 MYSQL . . . 72
8.18.2.3 SQLITE . . . 72
8.19 com.projectmanagement.lib.models.Task Class Reference . . . 72
8.19.1 Detailed Description . . . 73
8.19.2 Constructor & Destructor Documentation . . . 73
8.19.2.1 Task() [1/2] . . . 73
8.19.2.2 Task() [2/2] . . . 73
8.19.3 Member Function Documentation . . . 74
8.19.3.1 displayDetails() . . . 74
8.19.3.2 getAssignedUser() . . . 74
8.19.3.3 getDescription() . . . 74
8.19.3.4 getStatus() . . . 75
8.19.3.5 getTitle() . . . 75
8.19.3.6 setAssignedUser() . . . 75
8.19.3.7 setDescription() . . . 76
8.19.3.8 setStatus() . . . 76

8.19.3.9 setTitle() . . . 76
8.20 com.projectmanagement.lib.services.TaskService Class Reference . . . 77
8.20.1 Detailed Description . . . 77
8.20.2 Constructor & Destructor Documentation . . . 77
8.20.2.1 TaskService() . . . 77
8.20.3 Member Function Documentation . . . 77
8.20.3.1 assignTaskToUser() . . . 77
8.20.3.2 getAllTasks() . . . 78
8.20.3.3 registerTask() . . . 78
8.20.3.4 updateTaskStatus() . . . 79
8.21 com.projectmanagement.lib.models.TaskStatus Enum Reference . . . 80
8.21.1 Detailed Description . . . 80
8.21.2 Member Data Documentation . . . 80
8.21.2.1 DONE . . . 80
8.21.2.2 IN_PROGRESS . . . 80
8.21.2.3 TODO . . . 80
8.22 com.projectmanagement.lib.models.User Class Reference . . . 81
8.22.1 Detailed Description . . . 81
8.22.2 Constructor & Destructor Documentation . . . 81
8.22.2.1 User() [1/2] . . . 81
8.22.2.2 User() [2/2] . . . 82
8.22.3 Member Function Documentation . . . 82
8.22.3.1 displayDetails() . . . 82
8.22.3.2 getEmail() . . . 82
8.22.3.3 getPassword() . . . 83
8.22.3.4 getUsername() . . . 83
8.22.3.5 setEmail() . . . 83
8.22.3.6 setPassword() . . . 84
8.22.3.7 setUsername() . . . 84
8.23 com.projectmanagement.lib.services.UserService Class Reference . . . 84
8.23.1 Detailed Description . . . 84
8.23.2 Constructor & Destructor Documentation . . . 85
8.23.2.1 UserService() . . . 85
8.23.3 Member Function Documentation . . . 85
8.23.3.1 getAllUsers() . . . 85
8.23.3.2 login() . . . 85
8.23.3.3 registerUser() . . . 86

9 File Documentation **87**
9.1 projectmanager-app/src/main/java/com/projectmanagement/app/Main.java File Reference . . . 87
9.2 Main.java . . . 88
9.3 projectmanager-app/src/main/java/com/projectmanagement/lib/models/BaseEntity.java File Reference 93

9.4 BaseEntity.java . . . 94
9.5 projectmanager-app/src/main/java/com/projectmanagement/lib/models/Project.java File Reference . . 94
9.6 Project.java . . . 95
9.7 projectmanager-app/src/main/java/com/projectmanagement/lib/models/Task.java File Reference . . . 95
9.8 Task.java . . . 96
9.9 projectmanager-app/src/main/java/com/projectmanagement/lib/models/TaskStatus.java File Reference 97
9.10 TaskStatus.java . . . 97
9.11 projectmanager-app/src/main/java/com/projectmanagement/lib/models/User.java File Reference . . 97
9.12 User.java . . . 97
9.13 projectmanager-app/src/main/java/com/projectmanagement/lib/services/ProjectService.java File Reference . . . 98
9.14 ProjectService.java . . . 98
9.15 projectmanager-app/src/main/java/com/projectmanagement/lib/services/ReportService.java File Reference . . . 99
9.16 ReportService.java . . . 99
9.17 projectmanager-app/src/main/java/com/projectmanagement/lib/services/TaskService.java File Reference . . . 100
9.18 TaskService.java . . . 101
9.19 projectmanager-app/src/main/java/com/projectmanagement/lib/services/UserService.java File Reference . . . 101
9.20 UserService.java . . . 102
9.21 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/BinaryProjectRepository.java File Reference . . . 103
9.22 BinaryProjectRepository.java . . . 103
9.23 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/BinaryTaskRepository.java File Reference . . . 104
9.24 BinaryTaskRepository.java . . . 105
9.25 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/BinaryUserRepository.java File Reference . . . 106
9.26 BinaryUserRepository.java . . . 106
9.27 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/IRepository.java File Reference107
9.28 IRepository.java . . . 107
9.29 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/MySQLProjectRepository.java File Reference . . . 108
9.30 MySQLProjectRepository.java . . . 108
9.31 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/MySQLTaskRepository.java File Reference . . . 110
9.32 MySQLTaskRepository.java . . . 110
9.33 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/MySQLUserRepository.java File Reference . . . 112
9.34 MySQLUserRepository.java . . . 113
9.35 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/RepositoryFactory.java File Reference . . . 114
9.36 RepositoryFactory.java . . . 114

9.37 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/SQLiteProjectRepository.java File Reference . . . 115
9.38 SQLiteProjectRepository.java . . . 116
9.39 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/SQLiteTaskRepository.java File Reference . . . 118
9.40 SQLiteTaskRepository.java . . . 118
9.41 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/SQLiteUserRepository.java File Reference . . . 120
9.42 SQLiteUserRepository.java . . . 120
9.43 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/StorageConfig.java File Reference . . . 122
9.44 StorageConfig.java . . . 122
9.45 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/StorageType.java File Reference . . . 122
9.46 StorageType.java . . . 122

Chapter 1

Directory Hierarchy

1.1 Directories

app . . . 15
Main.java . . . 87
com . . . 15
projectmanagement . . . 16
app . . . 15
Main.java . . . 87
lib . . . 15
models . . . 16
BaseEntity.java . . . 93
Project.java . . . 94
Task.java . . . 95
TaskStatus.java . . . 97
User.java . . . 97
services . . . 16
ProjectService.java . . . 98
ReportService.java . . . 99
TaskService.java . . . 100
UserService.java . . . 101
storage . . . 17
BinaryProjectRepository.java . . . 103
BinaryTaskRepository.java . . . 104
BinaryUserRepository.java . . . 106
IRepository.java . . . 107
MySQLProjectRepository.java . . . 108
MySQLTaskRepository.java . . . 110
MySQLUserRepository.java . . . 112
RepositoryFactory.java . . . 114
SQLiteProjectRepository.java . . . 115
SQLiteTaskRepository.java . . . 118
SQLiteUserRepository.java . . . 120
StorageConfig.java . . . 122
StorageType.java . . . 122
java . . . 15
com . . . 15
projectmanagement . . . 16
app . . . 15
Main.java . . . 87

lib . . . 15
models . . . 16
BaseEntity.java . . . 93
Project.java . . . 94
Task.java . . . 95
TaskStatus.java . . . 97
User.java . . . 97
services . . . 16
ProjectService.java . . . 98
ReportService.java . . . 99
TaskService.java . . . 100
UserService.java . . . 101
storage . . . 17
BinaryProjectRepository.java . . . 103
BinaryTaskRepository.java . . . 104
BinaryUserRepository.java . . . 106
IRepository.java . . . 107
MySQLProjectRepository.java . . . 108
MySQLTaskRepository.java . . . 110
MySQLUserRepository.java . . . 112
RepositoryFactory.java . . . 114
SQLiteProjectRepository.java . . . 115
SQLiteTaskRepository.java . . . 118
SQLiteUserRepository.java . . . 120
StorageConfig.java . . . 122
StorageType.java . . . 122
lib . . . 15
models . . . 16
BaseEntity.java . . . 93
Project.java . . . 94
Task.java . . . 95
TaskStatus.java . . . 97
User.java . . . 97
services . . . 16
ProjectService.java . . . 98
ReportService.java . . . 99
TaskService.java . . . 100
UserService.java . . . 101
storage . . . 17
BinaryProjectRepository.java . . . 103
BinaryTaskRepository.java . . . 104
BinaryUserRepository.java . . . 106
IRepository.java . . . 107
MySQLProjectRepository.java . . . 108
MySQLTaskRepository.java . . . 110
MySQLUserRepository.java . . . 112
RepositoryFactory.java . . . 114
SQLiteProjectRepository.java . . . 115
SQLiteTaskRepository.java . . . 118
SQLiteUserRepository.java . . . 120
StorageConfig.java . . . 122
StorageType.java . . . 122
main . . . 16
java . . . 15
com . . . 15
projectmanagement . . . 16
app . . . 15

Main.java . . . 87
lib . . . 15
models . . . 16
BaseEntity.java . . . 93
Project.java . . . 94
Task.java . . . 95
TaskStatus.java . . . 97
User.java . . . 97
services . . . 16
ProjectService.java . . . 98
ReportService.java . . . 99
TaskService.java . . . 100
UserService.java . . . 101
storage . . . 17
BinaryProjectRepository.java . . . 103
BinaryTaskRepository.java . . . 104
BinaryUserRepository.java . . . 106
IRepository.java . . . 107
MySQLProjectRepository.java . . . 108
MySQLTaskRepository.java . . . 110
MySQLUserRepository.java . . . 112
RepositoryFactory.java . . . 114
SQLiteProjectRepository.java . . . 115
SQLiteTaskRepository.java . . . 118
SQLiteUserRepository.java . . . 120
StorageConfig.java . . . 122
StorageType.java . . . 122
models . . . 16
BaseEntity.java . . . 93
Project.java . . . 94
Task.java . . . 95
TaskStatus.java . . . 97
User.java . . . 97
projectmanagement . . . 16
app . . . 15
Main.java . . . 87
lib . . . 15
models . . . 16
BaseEntity.java . . . 93
Project.java . . . 94
Task.java . . . 95
TaskStatus.java . . . 97
User.java . . . 97
services . . . 16
ProjectService.java . . . 98
ReportService.java . . . 99
TaskService.java . . . 100
UserService.java . . . 101
storage . . . 17
BinaryProjectRepository.java . . . 103
BinaryTaskRepository.java . . . 104
BinaryUserRepository.java . . . 106
IRepository.java . . . 107
MySQLProjectRepository.java . . . 108
MySQLTaskRepository.java . . . 110
MySQLUserRepository.java . . . 112
RepositoryFactory.java . . . 114

SQLiteProjectRepository.java . . . 115
SQLiteTaskRepository.java . . . 118
SQLiteUserRepository.java . . . 120
StorageConfig.java . . . 122
StorageType.java . . . 122
projectmanager-app . . . 16
src . . . 17
main . . . 16
java . . . 15
com . . . 15
projectmanagement . . . 16
app . . . 15
Main.java . . . 87
lib . . . 15
models . . . 16
BaseEntity.java . . . 93
Project.java . . . 94
Task.java . . . 95
TaskStatus.java . . . 97
User.java . . . 97
services . . . 16
ProjectService.java . . . 98
ReportService.java . . . 99
TaskService.java . . . 100
UserService.java . . . 101
storage . . . 17
BinaryProjectRepository.java . . . 103
BinaryTaskRepository.java . . . 104
BinaryUserRepository.java . . . 106
IRepository.java . . . 107
MySQLProjectRepository.java . . . 108
MySQLTaskRepository.java . . . 110
MySQLUserRepository.java . . . 112
RepositoryFactory.java . . . 114
SQLiteProjectRepository.java . . . 115
SQLiteTaskRepository.java . . . 118
SQLiteUserRepository.java . . . 120
StorageConfig.java . . . 122
StorageType.java . . . 122
services . . . 16
ProjectService.java . . . 98
ReportService.java . . . 99
TaskService.java . . . 100
UserService.java . . . 101
src . . . 17
main . . . 16
java . . . 15
com . . . 15
projectmanagement . . . 16
app . . . 15
Main.java . . . 87
lib . . . 15
models . . . 16
BaseEntity.java . . . 93
Project.java . . . 94
Task.java . . . 95
TaskStatus.java . . . 97

User.java . . . 97
services . . . 16
ProjectService.java . . . 98
ReportService.java . . . 99
TaskService.java . . . 100
UserService.java . . . 101
storage . . . 17
BinaryProjectRepository.java . . . 103
BinaryTaskRepository.java . . . 104
BinaryUserRepository.java . . . 106
IRepository.java . . . 107
MySQLProjectRepository.java . . . 108
MySQLTaskRepository.java . . . 110
MySQLUserRepository.java . . . 112
RepositoryFactory.java . . . 114
SQLiteProjectRepository.java . . . 115
SQLiteTaskRepository.java . . . 118
SQLiteUserRepository.java . . . 120
StorageConfig.java . . . 122
StorageType.java . . . 122
storage . . . 17
BinaryProjectRepository.java . . . 103
BinaryTaskRepository.java . . . 104
BinaryUserRepository.java . . . 106
IRepository.java . . . 107
MySQLProjectRepository.java . . . 108
MySQLTaskRepository.java . . . 110
MySQLUserRepository.java . . . 112
RepositoryFactory.java . . . 114
SQLiteProjectRepository.java . . . 115
SQLiteTaskRepository.java . . . 118
SQLiteUserRepository.java . . . 120
StorageConfig.java . . . 122
StorageType.java . . . 122

Chapter 2

Namespace Index

2.1 Package List

Here are the packages with brief descriptions (if available):

com.projectmanagement.app 19
com.projectmanagement.lib.models 19
com.projectmanagement.lib.services 19
com.projectmanagement.lib.storage 20

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

com.projectmanagement.lib.storage.IRepository< T > . . . 32
com.projectmanagement.lib.storage.IRepository< Project > . . . 32
  com.projectmanagement.lib.storage.BinaryProjectRepository . . . 24
  com.projectmanagement.lib.storage.MySQLProjectRepository . . . 35
  com.projectmanagement.lib.storage.SQLiteProjectRepository . . . 58
com.projectmanagement.lib.storage.IRepository< Task > . . . 32
  com.projectmanagement.lib.storage.BinaryTaskRepository . . . 26
  com.projectmanagement.lib.storage.MySQLTaskRepository . . . 39
  com.projectmanagement.lib.storage.SQLiteTaskRepository . . . 62
com.projectmanagement.lib.storage.IRepository< User > . . . 32
  com.projectmanagement.lib.storage.BinaryUserRepository . . . 29
  com.projectmanagement.lib.storage.MySQLUserRepository . . . 43
  com.projectmanagement.lib.storage.SQLiteUserRepository . . . 66
com.projectmanagement.app.Main . . . 34
com.projectmanagement.lib.services.ProjectService . . . 52
com.projectmanagement.lib.services.ReportService . . . 54
com.projectmanagement.lib.storage.RepositoryFactory . . . 56
Serializable
  com.projectmanagement.lib.models.BaseEntity . . . 21
    com.projectmanagement.lib.models.Project . . . 47
    com.projectmanagement.lib.models.Task . . . 72
    com.projectmanagement.lib.models.User . . . 81
com.projectmanagement.lib.storage.StorageConfig . . . 70
com.projectmanagement.lib.storage.StorageType . . . 71
com.projectmanagement.lib.services.TaskService . . . 77
com.projectmanagement.lib.models.TaskStatus . . . 80
com.projectmanagement.lib.services.UserService . . . 84

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

com.projectmanagement.lib.models.BaseEntity . . . 21
com.projectmanagement.lib.storage.BinaryProjectRepository . . . 24
com.projectmanagement.lib.storage.BinaryTaskRepository . . . 26
com.projectmanagement.lib.storage.BinaryUserRepository . . . 29
com.projectmanagement.lib.storage.IRepository< T > . . . 32
com.projectmanagement.app.Main . . . 34
com.projectmanagement.lib.storage.MySQLProjectRepository . . . 35
com.projectmanagement.lib.storage.MySQLTaskRepository . . . 39
com.projectmanagement.lib.storage.MySQLUserRepository . . . 43
com.projectmanagement.lib.models.Project . . . 47
com.projectmanagement.lib.services.ProjectService . . . 52
com.projectmanagement.lib.services.ReportService . . . 54
com.projectmanagement.lib.storage.RepositoryFactory . . . 56
com.projectmanagement.lib.storage.SQLiteProjectRepository . . . 58
com.projectmanagement.lib.storage.SQLiteTaskRepository . . . 62
com.projectmanagement.lib.storage.SQLiteUserRepository . . . 66
com.projectmanagement.lib.storage.StorageConfig . . . 70
com.projectmanagement.lib.storage.StorageType . . . 71
com.projectmanagement.lib.models.Task . . . 72
com.projectmanagement.lib.services.TaskService . . . 77
com.projectmanagement.lib.models.TaskStatus . . . 80
com.projectmanagement.lib.models.User . . . 81
com.projectmanagement.lib.services.UserService . . . 84

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

projectmanager-app/src/main/java/com/projectmanagement/app/Main.java 87
projectmanager-app/src/main/java/com/projectmanagement/lib/models/BaseEntity.java 93
projectmanager-app/src/main/java/com/projectmanagement/lib/models/Project.java 94
projectmanager-app/src/main/java/com/projectmanagement/lib/models/Task.java 95
projectmanager-app/src/main/java/com/projectmanagement/lib/models/TaskStatus.java 97
projectmanager-app/src/main/java/com/projectmanagement/lib/models/User.java 97
projectmanager-app/src/main/java/com/projectmanagement/lib/services/ProjectService.java 98
projectmanager-app/src/main/java/com/projectmanagement/lib/services/ReportService.java 99
projectmanager-app/src/main/java/com/projectmanagement/lib/services/TaskService.java 100
projectmanager-app/src/main/java/com/projectmanagement/lib/services/UserService.java 101
projectmanager-app/src/main/java/com/projectmanagement/lib/storage/BinaryProjectRepository.java . . 103
projectmanager-app/src/main/java/com/projectmanagement/lib/storage/BinaryTaskRepository.java . . . 104
projectmanager-app/src/main/java/com/projectmanagement/lib/storage/BinaryUserRepository.java . . . 106
projectmanager-app/src/main/java/com/projectmanagement/lib/storage/IRepository.java 107
projectmanager-app/src/main/java/com/projectmanagement/lib/storage/MySQLProjectRepository.java . 108
projectmanager-app/src/main/java/com/projectmanagement/lib/storage/MySQLTaskRepository.java . . . 110
projectmanager-app/src/main/java/com/projectmanagement/lib/storage/MySQLUserRepository.java . . . 112
projectmanager-app/src/main/java/com/projectmanagement/lib/storage/RepositoryFactory.java 114
projectmanager-app/src/main/java/com/projectmanagement/lib/storage/SQLiteProjectRepository.java . . 115
projectmanager-app/src/main/java/com/projectmanagement/lib/storage/SQLiteTaskRepository.java . . . 118
projectmanager-app/src/main/java/com/projectmanagement/lib/storage/SQLiteUserRepository.java . . . 120
projectmanager-app/src/main/java/com/projectmanagement/lib/storage/StorageConfig.java 122
projectmanager-app/src/main/java/com/projectmanagement/lib/storage/StorageType.java 122

Chapter 6

Directory Documentation

6.1 projectmanager-app/src/main/java/com/projectmanagement/app Directory Reference

Files

- file Main.java

6.2 projectmanager-app/src/main/java/com Directory Reference

Directories

- directory projectmanagement

6.3 projectmanager-app/src/main/java Directory Reference

Directories

- directory com

6.4 projectmanager-app/src/main/java/com/projectmanagement/lib Directory Reference

Directories

- directory models
- directory services
- directory storage

6.5 projectmanager-app/src/main Directory Reference

Directories

- directory java

6.6 projectmanager-app/src/main/java/com/projectmanagement/lib/models Directory Reference

Files

- file BaseEntity.java
- file Project.java
- file Task.java
- file TaskStatus.java
- file User.java

6.7 projectmanager-app/src/main/java/com/projectmanagement Directory Reference

Directories

- directory app
- directory lib

6.8 projectmanager-app Directory Reference

Directories

- directory src

6.9 projectmanager-app/src/main/java/com/projectmanagement/lib/services Directory Reference

Files

- file ProjectService.java
- file ReportService.java
- file TaskService.java
- file UserService.java

6.10 projectmanager-app/src Directory Reference

Directories

- directory main

6.11 projectmanager-app/src/main/java/com/projectmanagement/lib/↩ storage Directory Reference

Files

- file BinaryProjectRepository.java
- file BinaryTaskRepository.java
- file BinaryUserRepository.java
- file IRepository.java
- file MySQLProjectRepository.java
- file MySQLTaskRepository.java
- file MySQLUserRepository.java
- file RepositoryFactory.java
- file SQLiteProjectRepository.java
- file SQLiteTaskRepository.java
- file SQLiteUserRepository.java
- file StorageConfig.java
- file StorageType.java

Chapter 7

Namespace Documentation

7.1 Package com.projectmanagement.app

Classes

- class Main

7.2 Package com.projectmanagement.lib.models

Classes

- class BaseEntity
- class Project
- class Task
- enum TaskStatus
- class User

7.3 Package com.projectmanagement.lib.services

Classes

- class ProjectService
- class ReportService
- class TaskService
- class UserService

7.4 Package com.projectmanagement.lib.storage

Classes

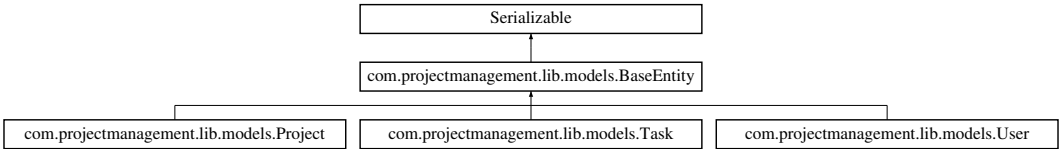
- class BinaryProjectRepository
- class BinaryTaskRepository
- class BinaryUserRepository
- interface IRepository
- class MySQLProjectRepository
- class MySQLTaskRepository
- class MySQLUserRepository
- class RepositoryFactory
- class SQLiteProjectRepository
- class SQLiteTaskRepository
- class SQLiteUserRepository
- class StorageConfig
- enum StorageType

Chapter 8

Class Documentation

8.1 com.projectmanagement.lib.models.BaseEntity Class Reference

Inheritance diagram for com.projectmanagement.lib.models.BaseEntity:



Public Member Functions

- BaseEntity ()
- BaseEntity (String id)
- String getId ()
- void setId (String id)
- LocalDateTime getCreatedDate ()
- void setCreatedDate (LocalDateTime createdDate)
- abstract void displayDetails ()

8.1.1 Detailed Description

Abstract base class for all domain models.

Applies the Abstraction and Inheritance principles of OOP. Provides common properties like id and createdDate for all entities.

Definition at line 10 of file BaseEntity.java.

8.1.2 Constructor & Destructor Documentation

8.1.2.1 BaseEntity() [1/2]

```
com.projectmanagement.lib.models.BaseEntity.BaseEntity ()
```

Default constructor for BaseEntity.

Initializes the createdDate to the current date and time.

Definition at line 29 of file BaseEntity.java.

```
00029                         {
00030         this.createdDate = LocalDateTime.now();
00031     }
```

8.1.2.2 BaseEntity() [2/2]

```
com.projectmanagement.lib.models.BaseEntity.BaseEntity (
            String id)
```

Parameterized constructor for BaseEntity.

Parameters

id	The unique identifier for the entity.

Definition at line 37 of file BaseEntity.java.

```
00037                                 {
00038         this();
00039         this.id = id;
00040     }
```

8.1.3 Member Function Documentation

8.1.3.1 displayDetails()

```
abstract void com.projectmanagement.lib.models.BaseEntity.displayDetails () [abstract]
```

Abstract method to display entity details.

Demonstrates Polymorphism. Each subclass must implement this method to provide specific details.

Reimplemented in com.projectmanagement.lib.models.Project, com.projectmanagement.lib.models.Task, and com.projectmanagement.lib.models.User.

8.1.3.2 getCreatedDate()

```
LocalDateTime com.projectmanagement.lib.models.BaseEntity.getCreatedDate ()
```

Gets the creation date of the entity.

Returns

The date and time the entity was created.

Definition at line 62 of file BaseEntity.java.

```
00062                                     {
00063         return createdDate;
00064     }
```

8.1.3.3 getId()

```
String com.projectmanagement.lib.models.BaseEntity.getId ()
```

Gets the unique identifier of the entity.

Returns

The entity's ID.

Definition at line 46 of file BaseEntity.java.

```
00046                         {
00047         return id;
00048     }
```

Referenced by com.projectmanagement.lib.models.Project.displayDetails(), com.projectmanagement.lib.models.Task.displayDetails() com.projectmanagement.lib.models.User.displayDetails(), com.projectmanagement.lib.storage.MySQLProjectRepository.save(), com.projectmanagement.lib.storage.MySQLTaskRepository.save(), com.projectmanagement.lib.storage.MySQLUserRepository.save(com.projectmanagement.lib.storage.SQLiteProjectRepository.save(), com.projectmanagement.lib.storage.SQLiteTaskRepository.save com.projectmanagement.lib.storage.SQLiteUserRepository.save(), com.projectmanagement.lib.storage.BinaryProjectRepository.upda com.projectmanagement.lib.storage.BinaryTaskRepository.update(), com.projectmanagement.lib.storage.BinaryUserRepository.upda com.projectmanagement.lib.storage.MySQLProjectRepository.update(), com.projectmanagement.lib.storage.MySQLTaskRepository.u com.projectmanagement.lib.storage.MySQLUserRepository.update(), com.projectmanagement.lib.storage.SQLiteProjectRepository.u com.projectmanagement.lib.storage.SQLiteTaskRepository.update(), and com.projectmanagement.lib.storage.SQLiteUserRepository.

8.1.3.4 setCreatedDate()

```
void com.projectmanagement.lib.models.BaseEntity.setCreatedDate (
            LocalDateTime createdDate)
```

Sets the creation date of the entity.

Parameters

createdDate	The new creation date.

Definition at line 70 of file BaseEntity.java.

```
00070                                                    {
00071         this.createdDate = createdDate;
00072     }
```

8.1.3.5 setId()

```
void com.projectmanagement.lib.models.BaseEntity.setId (
            String id)
```

Sets the unique identifier of the entity.

Parameters

id	The new ID to set.

Definition at line 54 of file BaseEntity.java.

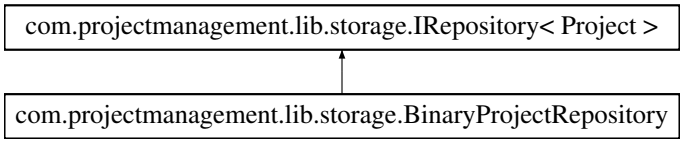
```
00054                                 {
00055         this.id = id;
00056     }
```

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/models/BaseEntity.java

8.2 com.projectmanagement.lib.storage.BinaryProjectRepository Class Reference

Inheritance diagram for com.projectmanagement.lib.storage.BinaryProjectRepository:



Public Member Functions

- void save (Project entity)
- Project findById (String id)
- List< Project > findAll ()
- void update (Project entity)
- void delete (String id)

Public Member Functions inherited from com.projectmanagement.lib.storage.IRepository< Project >

- void save (T entity)
- T findById (String id)
- List< T > findAll ()
- void update (T entity)
- void delete (String id)

8.2.1 Detailed Description

Binary File I/O implementation of the Project repository.

Implements IRepository to persist Project entities to local binary files.

Definition at line 12 of file BinaryProjectRepository.java.

8.2.2 Member Function Documentation

8.2.2.1 delete()

```
void com.projectmanagement.lib.storage.BinaryProjectRepository.delete (
            String id)
```

Eliminates a Project by ID from the binary data store.

Parameters

id	Distinct identifier of the target Project.

Definition at line 106 of file BinaryProjectRepository.java.

```
00106                                   {
00107         List<Project> projects = loadFromFile();
00108         projects.removeIf(p -> p.getId().equals(id));
00109         saveToFile(projects);
00110     }
```

8.2.2.2 findAll()

```
List< Project > com.projectmanagement.lib.storage.BinaryProjectRepository.findAll ()
```

Loads all stored Project entries from binary serialization.

Returns

Complete list of documented Projects.

Definition at line 81 of file BinaryProjectRepository.java.

```
00081                                   {
00082         return loadFromFile();
00083     }
```

8.2.2.3 findById()

```
Project com.projectmanagement.lib.storage.BinaryProjectRepository.findById (
            String id)
```

Reads a Project by ID from the binary file stream.

Parameters

id	The sought-after Project ID.

Returns

The Project object, or null on failure.

Definition at line 66 of file BinaryProjectRepository.java.

```
00066                                   {
00067         List<Project> projects = loadFromFile();
00068         for (Project p : projects) {
00069             if (p.getId().equals(id)) {
00070                 return p;
00071             }
00072         }
00073         return null;
00074     }
```

8.2.2.4 save()

```
void com.projectmanagement.lib.storage.BinaryProjectRepository.save (
            Project entity)
```

Writes a new Project entity into the binary file.

Parameters

entity	The Project instance to log.

Definition at line 54 of file BinaryProjectRepository.java.

```
00054                                       {
00055         List<Project> projects = loadFromFile();
00056         projects.add(entity);
00057         saveToFile(projects);
00058     }
```

8.2.2.5 update()

```
void com.projectmanagement.lib.storage.BinaryProjectRepository.update (
            Project entity)
```

Replaces an existing Project record in the binary file with new details.

Parameters

entity	Modified Project object.

Definition at line 90 of file BinaryProjectRepository.java.

```
00090                                       {
00091         List<Project> projects = loadFromFile();
00092         for (int i = 0; i < projects.size(); i++) {
00093             if (projects.get(i).getId().equals(entity.getId())) {
00094                 projects.set(i, entity);
00095                 break;
00096             }
00097         }
00098         saveToFile(projects);
00099     }
```

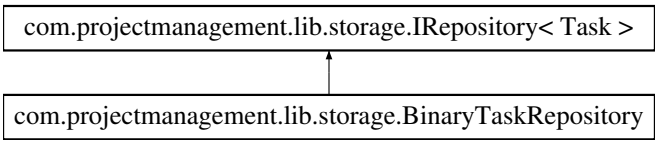
References com.projectmanagement.lib.models.BaseEntity.getId().

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/storage/BinaryProjectRepository.java

8.3 com.projectmanagement.lib.storage.BinaryTaskRepository Class Reference

Inheritance diagram for com.projectmanagement.lib.storage.BinaryTaskRepository:



Public Member Functions

- void save (Task entity)
- Task findById (String id)
- List< Task > findAll ()
- void update (Task entity)
- void delete (String id)

Public Member Functions inherited from com.projectmanagement.lib.storage.IRepository< Task >

- void save (T entity)
- T findById (String id)
- List< T > findAll ()
- void update (T entity)
- void delete (String id)

8.3.1 Detailed Description

Binary File I/O implementation of the Task repository.

Implements IRepository to persist Task entities to local binary file storage.

Definition at line 12 of file BinaryTaskRepository.java.

8.3.2 Member Function Documentation

8.3.2.1 delete()

```
void com.projectmanagement.lib.storage.BinaryTaskRepository.delete (
            String id)
```

Strips a Task entity from the binary footprint by ID matching.

Parameters

id	The corresponding identifier.

Definition at line 106 of file BinaryTaskRepository.java.

```
00106                                   {
00107         List<Task> tasks = loadFromFile();
00108         tasks.removeIf(t -> t.getId().equals(id));
00109         saveToFile(tasks);
00110     }
```

8.3.2.2 findAll()

```
List< Task > com.projectmanagement.lib.storage.BinaryTaskRepository.findAll ()
```

Interprets all Task entries secured inside binary files.

Returns

Collection array of Tasks.

Definition at line 81 of file BinaryTaskRepository.java.

```
00081                                   {
00082         return loadFromFile();
00083     }
```

8.3.2.3 findById()

```
Task com.projectmanagement.lib.storage.BinaryTaskRepository.findById (
            String id)
```

Fetches a Task by correlating ID from the binary data stream.

Parameters

id	The matching Task ID.

Returns

The target Task object, resolving to null otherwise.

Definition at line 66 of file BinaryTaskRepository.java.

```
00066                                   {
00067         List<Task> tasks = loadFromFile();
00068         for (Task t : tasks) {
00069             if (t.getId().equals(id)) {
00070                 return t;
00071             }
00072         }
00073         return null;
00074     }
```

8.3.2.4 save()

```
void com.projectmanagement.lib.storage.BinaryTaskRepository.save (
            Task entity)
```

Generates a new Task recording in the binary file.

Parameters

entity	The Task form to log.

Definition at line 54 of file BinaryTaskRepository.java.

```
00054                                   {
00055         List<Task> tasks = loadFromFile();
00056         tasks.add(entity);
00057         saveToFile(tasks);
00058     }
```

8.3.2.5 update()

```
void com.projectmanagement.lib.storage.BinaryTaskRepository.update (
            Task entity)
```

Refreshes an operational Task entity in binary file content.

Parameters

entity	Altered Task structure holding new details.

Definition at line 90 of file BinaryTaskRepository.java.

```
00090                                   {
00091         List<Task> tasks = loadFromFile();
00092         for (int i = 0; i < tasks.size(); i++) {
00093             if (tasks.get(i).getId().equals(entity.getId())) {
00094                 tasks.set(i, entity);
00095                 break;
00096             }
00097         }
00098         saveToFile(tasks);
00099     }
```

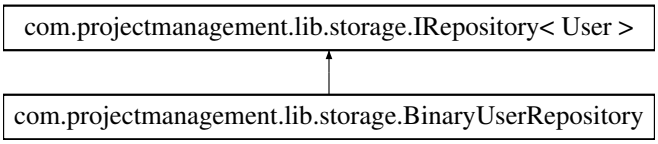
References com.projectmanagement.lib.models.BaseEntity.getId().

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/storage/BinaryTaskRepository.java

8.4 com.projectmanagement.lib.storage.BinaryUserRepository Class Reference

Inheritance diagram for com.projectmanagement.lib.storage.BinaryUserRepository:



Public Member Functions

- void save (User entity)
- User findById (String id)
- List< User > findAll ()
- void update (User entity)
- void delete (String id)

Public Member Functions inherited from com.projectmanagement.lib.storage.IRepository< User >

- void save (T entity)
- T findById (String id)
- List< T > findAll ()
- void update (T entity)
- void delete (String id)

8.4.1 Detailed Description

Binary File I/O implementation of the User repository.

Implements IRepository. Handles CRUD operations for User entities using a Binary File.

Definition at line 12 of file BinaryUserRepository.java.

8.4.2 Member Function Documentation

8.4.2.1 delete()

```
void com.projectmanagement.lib.storage.BinaryUserRepository.delete (
            String id)
```

Purges a User from the binary storage system by ID.

Parameters

id	The ID of the User to delete.

Definition at line 106 of file BinaryUserRepository.java.

```
00106                                   {
00107         List<User> users = loadFromFile();
00108         users.removeIf(u -> u.getId().equals(id));
00109         saveToFile(users);
00110     }
```

8.4.2.2 findAll()

```
List< User > com.projectmanagement.lib.storage.BinaryUserRepository.findAll ()
```

Retrieves a roster of all Users from the binary file.

Returns

List of all stored User entities.

Definition at line 81 of file BinaryUserRepository.java.

```
00081                                   {
00082         return loadFromFile();
00083     }
```

8.4.2.3 findById()

```
User com.projectmanagement.lib.storage.BinaryUserRepository.findById (
            String id)
```

Finds a User by corresponding ID.

Parameters

id	The unique identifier.

Returns

The matched User, or null if not found.

Definition at line 66 of file BinaryUserRepository.java.

```
00066                                   {
00067         List<User> users = loadFromFile();
00068         for (User u : users) {
00069             if (u.getId().equals(id)) {
00070                 return u;
00071             }
00072         }
00073         return null;
00074     }
```

8.4.2.4 save()

```
void com.projectmanagement.lib.storage.BinaryUserRepository.save (
            User entity)
```

Creates a new User record in the binary file.

Parameters

entity	The User object to save.

Definition at line 54 of file BinaryUserRepository.java.

```
00054                                   {
00055         List<User> users = loadFromFile();
00056         users.add(entity);
00057         saveToFile(users);
00058     }
```

8.4.2.5 update()

```
void com.projectmanagement.lib.storage.BinaryUserRepository.update (
            User entity)
```

Overwrites an existing User's data in the binary file.

Parameters

entity	The updated User instance containing changes.

Definition at line 90 of file BinaryUserRepository.java.

```
00090                                   {
00091         List<User> users = loadFromFile();
00092         for (int i = 0; i < users.size(); i++) {
00093             if (users.get(i).getId().equals(entity.getId())) {
00094                 users.set(i, entity);
00095                 break;
00096             }
00097         }
00098         saveToFile(users);
00099     }
```

References com.projectmanagement.lib.models.BaseEntity.getId().

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/storage/BinaryUserRepository.java

8.5 com.projectmanagement.lib.storage.IRepository< T > Interface Template Reference

Public Member Functions

- void save (T entity)
- T findById (String id)
- List< T > findAll ()
- void update (T entity)
- void delete (String id)

8.5.1 Detailed Description

Generic repository interface for CRUD operations.

Parameters

<T>	The type of entity the repository manages.

Applies the Abstraction principle. Specifies standard behaviors for all storage types.

Definition at line 10 of file IRepository.java.

8.5.2 Member Function Documentation

8.5.2.1 delete()

```
void com.projectmanagement.lib.storage.IRepository< T >.delete (
            String id)
```

Deletes an entity from the repository based on its unique identifier.

Parameters

id	The unique identifier of the entity to be deleted.

8.5.2.2 findAll()

```
List< T > com.projectmanagement.lib.storage.IRepository< T >.findAll ()
```

Retrieves all entities from the repository.

Returns

A list containing all the entities.

8.5.2.3 findById()

```
T com.projectmanagement.lib.storage.IRepository< T >.findById (
            String id)
```

Retrieves an entity by its unique identifier.

Parameters

id	The unique identifier of the entity.

Returns

The entity object if found, null otherwise.

8.5.2.4 save()

```
void com.projectmanagement.lib.storage.IRepository< T >.save (
            T entity)
```

Creates a new entity in the repository.

Parameters

entity	The entity object to be saved.

8.5.2.5 update()

```
void com.projectmanagement.lib.storage.IRepository< T >.update (
            T entity)
```

Updates an existing entity in the repository.

Parameters

entity	The entity object containing the updated information.

The documentation for this interface was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/storage/IRepository.java

8.6 com.projectmanagement.app.Main Class Reference

Static Public Member Functions

- static void main (String[ ] args)

8.6.1 Detailed Description

Main entry point for the Project Management Console App UI.

Implements a standard console-based interface using Scanner.

Definition at line 23 of file Main.java.

8.6.2 Member Function Documentation

8.6.2.1 main()

```
void com.projectmanagement.app.Main.main (
            String[] args)  [static]
```

Bootstrapping function serving as application lifecycle container. Starts Terminal integrations and enters the eternal authentication/main-menu loop.

Parameters

args	Standalone command-line runtime instruction variables.

Definition at line 60 of file Main.java.

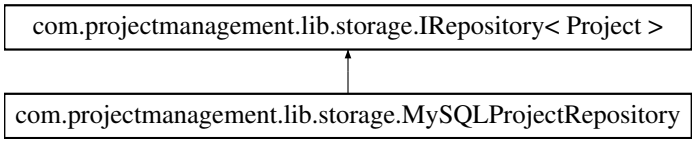
```
00060                                                   {
00061         // Initialize scanner freshly. Allows unit tests to intercept System.in cleanly.
00062         scanner = new Scanner(System.in);
00063
00064         try {
00065             terminal = TerminalBuilder.builder().jna(true).system(true).build();
00066         } catch (Exception e) {
00067             terminal = null;
00068         }
00069
00070         boolean appRunning = true;
00071         while (appRunning) {
00072             if (loggedInUser == null && !isGuest) {
00073                 appRunning = renderAuthMenu();
00074             } else {
00075                 appRunning = renderMainMenu();
00076             }
00077         }
00078         System.out.println("Exiting application. Goodbye!");
00079         scanner.close();
00080     }
```

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/app/Main.java

8.7 com.projectmanagement.lib.storage.MySQLProjectRepository Class Reference

Inheritance diagram for com.projectmanagement.lib.storage.MySQLProjectRepository:



Public Member Functions

- MySQLProjectRepository ()
- void save (Project entity)
- Project findById (String id)
- List< Project > findAll ()
- void update (Project entity)
- void delete (String id)

Public Member Functions inherited from com.projectmanagement.lib.storage.IRepository< Project >

- void save (T entity)
- T findById (String id)
- List< T > findAll ()
- void update (T entity)
- void delete (String id)

Protected Member Functions

- Connection getConnection () throws SQLException

8.7.1 Detailed Description

MySQL implementation of the Project repository.

Implements IRepository.

Definition at line 12 of file MySQLProjectRepository.java.

8.7.2 Constructor & Destructor Documentation

8.7.2.1 MySQLProjectRepository()

```
com.projectmanagement.lib.storage.MySQLProjectRepository.MySQLProjectRepository ()
```

Constructor for MySQLProjectRepository.

Definition at line 21 of file MySQLProjectRepository.java.

```
00021                                   {
00022         createTableIfNotExists();
00023     }
```

8.7.3 Member Function Documentation

8.7.3.1 delete()

```
void com.projectmanagement.lib.storage.MySQLProjectRepository.delete (
            String id)
```

Eradicates a Project from the MySQL database stream by ID.

Parameters

id	Targeted Project identifier.

Definition at line 177 of file MySQLProjectRepository.java.

```
00177                                   {
00178         String sql = "DELETE FROM projects WHERE id = ?";
00179         try (Connection conn = getConnection();
00180              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00181             pstmt.setString(1, id);
00182             pstmt.executeUpdate();
00183
00184             String delSql = "DELETE FROM project_tasks WHERE project_id = ?";
00185             try (PreparedStatement delStmt = conn.prepareStatement(delSql)) {
00186                 delStmt.setString(1, id);
00187                 delStmt.executeUpdate();
00188             }
00189         } catch (SQLException e) {
00190             System.err.println("Error deleting project: " + e.getMessage());
00191         }
00192     }
```

References getConnection().

8.7.3.2 findAll()

```
List< Project > com.projectmanagement.lib.storage.MySQLProjectRepository.findAll ()
```

Retrieves the entirety of Project records from the MySQL db.

Returns

Complete list of Project occurrences.

Definition at line 131 of file MySQLProjectRepository.java.

```
00131                                   {
00132         List<Project> projects = new ArrayList<>();
00133         String sql = "SELECT * FROM projects";
00134         try (Connection conn = getConnection();
00135              Statement stmt = conn.createStatement();
00136              ResultSet rs = stmt.executeQuery(sql)) {
00137             while (rs.next()) {
00138                 Project project = new Project(
00139                     rs.getString("id"),
00140                     rs.getString("name"),
00141                     rs.getString("description")
00142                 );
00143                 loadProjectTasks(conn, project);
00144                 projects.add(project);
00145             }
00146         } catch (SQLException e) {
00147             System.err.println("Error finding all projects: " + e.getMessage());
00148         }
00149         return projects;
00150     }
```

References getConnection().

8.7.3.3 findById()

```
Project com.projectmanagement.lib.storage.MySQLProjectRepository.findById (
            String id)
```

Looks up a Project from the MySQL db based on ID.

Parameters

id	The sought-after Project ID.

Returns

The distinct Project, or null if undiscovered.

Definition at line 90 of file MySQLProjectRepository.java.

```
00090                                       {
00091         String sql = "SELECT * FROM projects WHERE id = ?";
00092         try (Connection conn = getConnection();
00093              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00094             pstmt.setString(1, id);
00095             ResultSet rs = pstmt.executeQuery();
00096             if (rs.next()) {
00097                 Project project = new Project(
00098                     rs.getString("id"),
00099                     rs.getString("name"),
00100                     rs.getString("description")
00101                 );
00102                 loadProjectTasks(conn, project);
00103                 return project;
00104             }
00105         } catch (SQLException e) {
00106             System.err.println("Error finding project by id: " + e.getMessage());
00107         }
00108         return null;
00109     }
```

References getConnection().

8.7.3.4 getConnection()

```
Connection com.projectmanagement.lib.storage.MySQLProjectRepository.getConnection () throws
SQLException [protected]
```

Definition at line 194 of file MySQLProjectRepository.java.

```
00194 { return DriverManager.getConnection(URL, USER_DB, PASS_DB); }
```

Referenced by delete(), findAll(), findById(), save(), and update().

8.7.3.5 save()

```
void com.projectmanagement.lib.storage.MySQLProjectRepository.save (
            Project entity)
```

Inserts a new Project record into the MySQL db.

Parameters

entity	The Project instance to be added.

Definition at line 50 of file MySQLProjectRepository.java.

```
00050                                   {
00051         String sql = "INSERT INTO projects(id, name, description) VALUES(?, ?, ?)";
00052         try (Connection conn = getConnection();
00053              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00054             pstmt.setString(1, entity.getId());
00055             pstmt.setString(2, entity.getName());
00056             pstmt.setString(3, entity.getDescription());
00057             pstmt.executeUpdate();
00058
00059             saveProjectTasks(conn, entity);
00060         } catch (SQLException e) {
00061             System.err.println("Error creating project: " + e.getMessage());
00062         }
00063     }
```

References getConnection(), com.projectmanagement.lib.models.Project.getDescription(), com.projectmanagement.lib.models.BaseE
and com.projectmanagement.lib.models.Project.getName().

8.7.3.6 update()

```
void com.projectmanagement.lib.storage.MySQLProjectRepository.update (
            Project entity)
```

Applies modifications to a documented Project in the MySQL db.

Parameters

entity	Project entity carrying current revisions.

Definition at line 157 of file MySQLProjectRepository.java.

```
00157                                   {
00158         String sql = "UPDATE projects SET name = ?, description = ? WHERE id = ?";
00159         try (Connection conn = getConnection();
00160              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00161             pstmt.setString(1, entity.getName());
00162             pstmt.setString(2, entity.getDescription());
```

```
            pstmt.setString(3, entity.getId());
            pstmt.executeUpdate();

            saveProjectTasks(conn, entity);
        } catch (SQLException e) {
            System.err.println("Error updating project: " + e.getMessage());
        }
    }
```

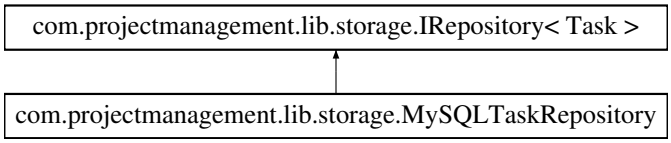
References getConnection(), com.projectmanagement.lib.models.Project.getDescription(), com.projectmanagement.lib.models.BaseE
and com.projectmanagement.lib.models.Project.getName().

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/storage/MySQLProjectRepository.java

8.8 com.projectmanagement.lib.storage.MySQLTaskRepository Class Reference

Inheritance diagram for com.projectmanagement.lib.storage.MySQLTaskRepository:



Public Member Functions

- MySQLTaskRepository ()
- void save (Task entity)
- Task findById (String id)
- List< Task > findAll ()
- void update (Task entity)
- void delete (String id)

Public Member Functions inherited from com.projectmanagement.lib.storage.IRepository< Task >

- void save (T entity)
- T findById (String id)
- List< T > findAll ()
- void update (T entity)
- void delete (String id)

Protected Member Functions

- Connection getConnection () throws SQLException

8.8.1 Detailed Description

MySQL implementation of the Task repository.

Implements IRepository.

Definition at line 14 of file MySQLTaskRepository.java.

8.8.2 Constructor & Destructor Documentation

8.8.2.1 MySQLTaskRepository()

```
com.projectmanagement.lib.storage.MySQLTaskRepository.MySQLTaskRepository ()
```

Constructor for MySQLTaskRepository.

Definition at line 23 of file MySQLTaskRepository.java.

```
00023                                   {
00024         createTableIfNotExists();
00025     }
```

8.8.3 Member Function Documentation

8.8.3.1 delete()

```
void com.projectmanagement.lib.storage.MySQLTaskRepository.delete (
            String id)
```

Withdraws a given Task from the associated MySQL db using ID.

Parameters

id	The corresponding Task id string.

Definition at line 151 of file MySQLTaskRepository.java.

```
00151                                   {
00152         String sql = "DELETE FROM tasks WHERE id = ?";
00153         try (Connection conn = getConnection();
00154              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00155             pstmt.setString(1, id);
00156             pstmt.executeUpdate();
00157         } catch (SQLException e) {
00158             System.err.println("Error deleting task: " + e.getMessage());
00159         }
00160     }
```

References getConnection().

8.8.3.2 findAll()

```
List< Task > com.projectmanagement.lib.storage.MySQLTaskRepository.findAll ()
```

Discovers and yields all Task forms positioned in the MySQL db.

Returns

Exhaustive list collection of Tasks.

Definition at line 100 of file MySQLTaskRepository.java.

```
00100                                   {
00101         List<Task> tasks = new ArrayList<>();
00102         String sql = "SELECT * FROM tasks";
00103         try (Connection conn = getConnection();
00104              Statement stmt = conn.createStatement();
00105              ResultSet rs = stmt.executeQuery(sql)) {
00106             MySQLUserRepository userRepo = new MySQLUserRepository();
00107             while (rs.next()) {
00108                 Task task = new Task(
00109                     rs.getString("id"),
00110                     rs.getString("title"),
00111                     rs.getString("description")
00112                 );
00113                 task.setStatus(TaskStatus.valueOf(rs.getString("status")));
00114                 String userId = rs.getString("assigned_user_id");
00115                 if (userId != null) {
00116                     task.setAssignedUser(userRepo.findById(userId));
00117                 }
00118                 tasks.add(task);
00119             }
00120         } catch (SQLException e) {
00121             System.err.println("Error finding all tasks: " + e.getMessage());
00122         }
00123         return tasks;
00124     }
```

References com.projectmanagement.lib.storage.MySQLUserRepository.findById(), getConnection(), com.projectmanagement.lib.mod
and com.projectmanagement.lib.models.Task.setStatus().

8.8.3.3 findById()

```
Task com.projectmanagement.lib.storage.MySQLTaskRepository.findById (
            String id)
```

Reads a specific Task from the MySQL db using its ID.

Parameters

id	Tracking index of the desired Task.

Returns

Single Task reference, or null when undiscovered.

Definition at line 69 of file MySQLTaskRepository.java.

```
00069                                   {
00070         String sql = "SELECT * FROM tasks WHERE id = ?";
00071         try (Connection conn = getConnection();
00072              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00073             pstmt.setString(1, id);
00074             ResultSet rs = pstmt.executeQuery();
00075             if (rs.next()) {
```

```
                Task task = new Task(
                    rs.getString("id"),
                    rs.getString("title"),
                    rs.getString("description")
                );
                task.setStatus(TaskStatus.valueOf(rs.getString("status")));
                String userId = rs.getString("assigned_user_id");
                if (userId != null) {
                    User assignee = new MySQLUserRepository().findById(userId);
                    task.setAssignedUser(assignee);
                }
                return task;
            }
        } catch (SQLException e) {
            System.err.println("Error finding task by id: " + e.getMessage());
        }
        return null;
    }
```

References com.projectmanagement.lib.storage.MySQLUserRepository.findById(), getConnection(), com.projectmanagement.lib.moc
and com.projectmanagement.lib.models.Task.setStatus().

8.8.3.4 getConnection()

```
Connection com.projectmanagement.lib.storage.MySQLTaskRepository.getConnection () throws SQLException
[protected]
```

Definition at line 162 of file MySQLTaskRepository.java.

```
00162 { return DriverManager.getConnection(URL, USER_DB, PASS_DB); }
```

Referenced by delete(), findAll(), findById(), save(), and update().

8.8.3.5 save()

```
void com.projectmanagement.lib.storage.MySQLTaskRepository.save (
            Task entity)
```

Initiates a Task entity registry within the MySQL db.

Parameters

entity	The Task implementation object.

Definition at line 48 of file MySQLTaskRepository.java.

```
00048                                   {
00049         String sql = "INSERT INTO tasks(id, title, description, status, assigned_user_id) VALUES(?, ?,
      ?, ?, ?)";
00050         try (Connection conn = getConnection();
00051              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00052             pstmt.setString(1, entity.getId());
00053             pstmt.setString(2, entity.getTitle());
00054             pstmt.setString(3, entity.getDescription());
00055             pstmt.setString(4, entity.getStatus().name());
00056             pstmt.setString(5, entity.getAssignedUser() != null ? entity.getAssignedUser().getId() :
      null);
00057             pstmt.executeUpdate();
00058         } catch (SQLException e) {
00059             System.err.println("Error creating task: " + e.getMessage());
00060         }
00061     }
```

References com.projectmanagement.lib.models.Task.getAssignedUser(), getConnection(), com.projectmanagement.lib.models.Task.g
com.projectmanagement.lib.models.BaseEntity.getId(), com.projectmanagement.lib.models.Task.getStatus(), and
com.projectmanagement.lib.models.Task.getTitle().

8.8.3.6 update()

```
void com.projectmanagement.lib.storage.MySQLTaskRepository.update (
            Task entity)
```

Re-registers an associated Task inside MySQL with incoming tweaks.

Parameters

entity	Modified Task form.

Definition at line 131 of file MySQLTaskRepository.java.

```
00131                                   {
00132         String sql = "UPDATE tasks SET title = ?, description = ?, status = ?, assigned_user_id = ?
      WHERE id = ?";
00133         try (Connection conn = getConnection();
00134              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00135             pstmt.setString(1, entity.getTitle());
00136             pstmt.setString(2, entity.getDescription());
00137             pstmt.setString(3, entity.getStatus().name());
00138             pstmt.setString(4, entity.getAssignedUser() != null ? entity.getAssignedUser().getId() :
      null);
00139             pstmt.setString(5, entity.getId());
00140             pstmt.executeUpdate();
00141         } catch (SQLException e) {
00142             System.err.println("Error updating task: " + e.getMessage());
00143         }
00144     }
```

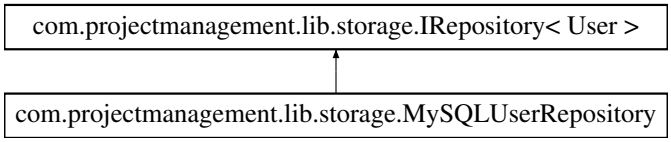
References com.projectmanagement.lib.models.Task.getAssignedUser(), getConnection(), com.projectmanagement.lib.models.Task.g com.projectmanagement.lib.models.BaseEntity.getId(), com.projectmanagement.lib.models.Task.getStatus(), and com.projectmanagement.lib.models.Task.getTitle().

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/storage/MySQLTaskRepository.java

8.9 com.projectmanagement.lib.storage.MySQLUserRepository Class Reference

Inheritance diagram for com.projectmanagement.lib.storage.MySQLUserRepository:



Public Member Functions

- MySQLUserRepository ()
- void save (User entity)
- User findById (String id)
- List< User > findAll ()
- void update (User entity)
- void delete (String id)

Public Member Functions inherited from com.projectmanagement.lib.storage.IRepository< User >

- void save (T entity)
- T findById (String id)
- List< T > findAll ()
- void update (T entity)
- void delete (String id)

Protected Member Functions

- Connection getConnection () throws SQLException

8.9.1 Detailed Description

MySQL implementation of the User repository.

Implements IRepository. Connects to a remote MySQL database for User CRUD operations.

Definition at line 12 of file MySQLUserRepository.java.

8.9.2 Constructor & Destructor Documentation

8.9.2.1 MySQLUserRepository()

```
com.projectmanagement.lib.storage.MySQLUserRepository.MySQLUserRepository ()
```

Constructor for MySQLUserRepository.

Definition at line 21 of file MySQLUserRepository.java.

```
00021                                   {
00022         createTableIfNotExists();
00023     }
```

8.9.3 Member Function Documentation

8.9.3.1 delete()

```
void com.projectmanagement.lib.storage.MySQLUserRepository.delete (
            String id)
```

Executes a DELETE statement on a User in the MySQL db.

Parameters

id	Target ID to drop.

Definition at line 134 of file MySQLUserRepository.java.

```
00134                                   {
00135         String sql = "DELETE FROM users WHERE id = ?";
00136         try (Connection conn = getConnection();
00137              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00138             pstmt.setString(1, id);
00139             pstmt.executeUpdate();
00140         } catch (SQLException e) {
00141             System.err.println("Error deleting user (MySQL): " + e.getMessage());
00142         }
00143     }
```

References getConnection().

8.9.3.2 findAll()

```
List< User > com.projectmanagement.lib.storage.MySQLUserRepository.findAll ()
```

Pulls all Users from the MySQL tables.

Returns

Full list of User entities.

Definition at line 90 of file MySQLUserRepository.java.

```
00090                                   {
00091         List<User> users = new ArrayList<>();
00092         String sql = "SELECT * FROM users";
00093         try (Connection conn = getConnection();
00094              Statement stmt = conn.createStatement();
00095              ResultSet rs = stmt.executeQuery(sql)) {
00096             while (rs.next()) {
00097                 users.add(new User(
00098                     rs.getString("id"),
00099                     rs.getString("username"),
00100                     rs.getString("email"),
00101                     rs.getString("password")
00102                 ));
00103             }
00104         } catch (SQLException e) {
00105             System.err.println("Error finding all users (MySQL): " + e.getMessage());
00106         }
00107         return users;
00108     }
```

References getConnection().

8.9.3.3 findById()

```
User com.projectmanagement.lib.storage.MySQLUserRepository.findById (
            String id)
```

Searches the MySQL db for a given User ID.

Parameters

id	The target User identifier.

Returns

The matched User, or null if unlocated.

Definition at line 65 of file MySQLUserRepository.java.

```
00065                                       {
00066         String sql = "SELECT * FROM users WHERE id = ?";
00067         try (Connection conn = getConnection();
00068              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00069             pstmt.setString(1, id);
00070             ResultSet rs = pstmt.executeQuery();
00071             if (rs.next()) {
00072                 return new User(
00073                     rs.getString("id"),
00074                     rs.getString("username"),
00075                     rs.getString("email"),
00076                     rs.getString("password")
00077                 );
00078             }
00079         } catch (SQLException e) {
00080             System.err.println("Error finding user by id (MySQL): " + e.getMessage());
00081         }
00082         return null;
00083     }
```

References getConnection().

Referenced by com.projectmanagement.lib.storage.MySQLTaskRepository.findAll(), and com.projectmanagement.lib.storage.MySQLT

8.9.3.4 getConnection()

```
Connection com.projectmanagement.lib.storage.MySQLUserRepository.getConnection () throws SQLException
[protected]
```

Definition at line 145 of file MySQLUserRepository.java.

```
00145 { return DriverManager.getConnection(URL, USER_DB, PASS_DB); }
```

Referenced by delete(), findAll(), findById(), save(), and update().

8.9.3.5 save()

```
void com.projectmanagement.lib.storage.MySQLUserRepository.save (
            User entity)
```

Persists a novel User entity to the MySQL db.

Parameters

entity	The User to save.

Definition at line 45 of file MySQLUserRepository.java.

```
00045                                   {
00046         String sql = "INSERT INTO users(id, username, email, password) VALUES(?, ?, ?, ?)";
00047         try (Connection conn = getConnection();
00048              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00049             pstmt.setString(1, entity.getId());
00050             pstmt.setString(2, entity.getUsername());
00051             pstmt.setString(3, entity.getEmail());
00052             pstmt.setString(4, entity.getPassword());
00053             pstmt.executeUpdate();
00054         } catch (SQLException e) {
00055             System.err.println("Error creating user (MySQL): " + e.getMessage());
00056         }
00057     }
```

References getConnection(), com.projectmanagement.lib.models.User.getEmail(), com.projectmanagement.lib.models.BaseEntity.get
com.projectmanagement.lib.models.User.getPassword(), and com.projectmanagement.lib.models.User.getUsername().

8.9.3.6 update()

```
void com.projectmanagement.lib.storage.MySQLUserRepository.update (
            User entity)
```

Merges an updated User entity into the MySQL db.

Parameters

entity	User object sporting recent updates.

Definition at line 115 of file MySQLUserRepository.java.

```
00115                                   {
00116         String sql = "UPDATE users SET username = ?, email = ?, password = ? WHERE id = ?";
00117         try (Connection conn = getConnection();
00118              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00119             pstmt.setString(1, entity.getUsername());
00120             pstmt.setString(2, entity.getEmail());
00121             pstmt.setString(3, entity.getPassword());
```

```
            pstmt.setString(4, entity.getId());
            pstmt.executeUpdate();
        } catch (SQLException e) {
            System.err.println("Error updating user (MySQL): " + e.getMessage());
        }
    }
```

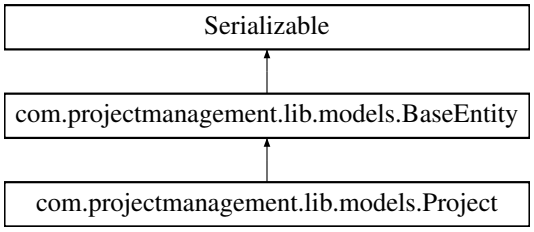
References getConnection(), com.projectmanagement.lib.models.User.getEmail(), com.projectmanagement.lib.models.BaseEntity.get
com.projectmanagement.lib.models.User.getPassword(), and com.projectmanagement.lib.models.User.getUsername().

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/storage/MySQLUserRepository.java

8.10 com.projectmanagement.lib.models.Project Class Reference

Inheritance diagram for com.projectmanagement.lib.models.Project:



Public Member Functions

- Project ()
- Project (String id, String name, String description)
- String getName ()
- void setName (String name)
- String getDescription ()
- void setDescription (String description)
- List< Task > getTasks ()
- void setTasks (List< Task > tasks)
- void addTask (Task task)
- void removeTask (Task task)
- void displayDetails ()

Public Member Functions inherited from com.projectmanagement.lib.models.BaseEntity

- BaseEntity ()
- BaseEntity (String id)
- String getId ()
- void setId (String id)
- LocalDateTime getCreatedDate ()
- void setCreatedDate (LocalDateTime createdDate)

8.10.1 Detailed Description

Represents a project in the system.

Inherits from BaseEntity. Contains a collection of tasks and assigned users, demonstrating object relationships and encapsulation.

Definition at line 10 of file Project.java.

8.10.2 Constructor & Destructor Documentation

8.10.2.1 Project() [1/2]

```
com.projectmanagement.lib.models.Project.Project ()
```

Default constructor for Project.

Initializes the tasks list.

Definition at line 30 of file Project.java.

```
00030                    {
00031        super();
00032        this.tasks = new ArrayList<>();
00033    }
```

8.10.2.2 Project() [2/2]

```
com.projectmanagement.lib.models.Project.Project (
            String id,
            String name,
            String description)
```

Parameterized constructor for Project.

Parameters

id	The unique identifier for the project.
name	The name of the project.
description	The description of the project.

Definition at line 41 of file Project.java.

```
00041                                                    {
00042        super(id);
00043        this.name = name;
00044        this.description = description;
00045        this.tasks = new ArrayList<>();
00046    }
```

8.10.3 Member Function Documentation

8.10.3.1 addTask()

```
void com.projectmanagement.lib.models.Project.addTask (
            Task task)
```

Adds a task to the project.

Parameters

task	The task to add.

Definition at line 100 of file Project.java.

```
00100                                   {
00101         this.tasks.add(task);
00102     }
```

Referenced by com.projectmanagement.lib.services.ProjectService.addTaskToProject().

8.10.3.2 displayDetails()

```
void com.projectmanagement.lib.models.Project.displayDetails ()
```

Displays the details of the project.

Implementation of the abstract method from BaseEntity. Shows polymorphism.

Reimplemented from com.projectmanagement.lib.models.BaseEntity.

Definition at line 117 of file Project.java.

```
00117                                   {
00118         System.out.println("Project ID: " + getId() + ", Name: " + name);
00119         System.out.println("Description: " + description);
00120         System.out.println("Total Tasks: " + tasks.size());
00121     }
```

References com.projectmanagement.lib.models.BaseEntity.getId().

8.10.3.3 getDescription()

```
String com.projectmanagement.lib.models.Project.getDescription ()
```

Gets the description of the project.

Returns

The project description.

Definition at line 68 of file Project.java.

```
00068                                   {
00069         return description;
00070     }
```

Referenced by com.projectmanagement.lib.storage.MySQLProjectRepository.save(), com.projectmanagement.lib.storage.SQLiteProje
com.projectmanagement.lib.storage.MySQLProjectRepository.update(), and com.projectmanagement.lib.storage.SQLiteProjectRepos

8.10.3.4 getName()

```
String com.projectmanagement.lib.models.Project.getName ()
```

Gets the name of the project.

Returns

The project name.

Definition at line 52 of file Project.java.

```
00052                                {
00053         return name;
00054     }
```

Referenced by com.projectmanagement.lib.storage.MySQLProjectRepository.save(), com.projectmanagement.lib.storage.SQLiteProje
com.projectmanagement.lib.storage.MySQLProjectRepository.update(), and com.projectmanagement.lib.storage.SQLiteProjectRepos

8.10.3.5 getTasks()

```
List< Task > com.projectmanagement.lib.models.Project.getTasks ()
```

Gets the list of tasks in the project.

Returns

The list of tasks.

Definition at line 84 of file Project.java.

```
00084                                    {
00085         return tasks;
00086     }
```

8.10.3.6 removeTask()

```
void com.projectmanagement.lib.models.Project.removeTask (
            Task task)
```

Removes a task from the project.

Parameters

task	The task to remove.

Definition at line 108 of file Project.java.

```
00108                                        {
00109         this.tasks.remove(task);
00110     }
```

8.10.3.7 setDescription()

```
void com.projectmanagement.lib.models.Project.setDescription (
            String description)
```

Sets the description of the project.

Parameters

description	The new project description.

Definition at line 76 of file Project.java.

```
00076                                                   {
00077         this.description = description;
00078     }
```

8.10.3.8 setName()

```
void com.projectmanagement.lib.models.Project.setName (
            String name)
```

Sets the name of the project.

Parameters

name	The new project name.

Definition at line 60 of file Project.java.

```
00060                                       {
00061         this.name = name;
00062     }
```

8.10.3.9 setTasks()

```
void com.projectmanagement.lib.models.Project.setTasks (
            List< Task > tasks)
```

Sets the list of tasks in the project.

Parameters

tasks	The new list of tasks.

Definition at line 92 of file Project.java.

```
00092                                           {
00093         this.tasks = tasks;
00094     }
```

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/models/Project.java

8.11 com.projectmanagement.lib.services.ProjectService Class Reference

Public Member Functions

- ProjectService (IRepository< Project > projectRepository)
- void createProject (String id, String name, String description)
- void addTaskToProject (String projectId, Task task)
- List< Project > getAllProjects ()

8.11.1 Detailed Description

Manages business operations and logic connected to Projects.

Responsible for core actions like creating a project and orchestrating modifications connected specifically to project data sets. Follows the Dependency Injection parameter layout.

Definition at line 12 of file ProjectService.java.

8.11.2 Constructor & Destructor Documentation

8.11.2.1 ProjectService()

```
com.projectmanagement.lib.services.ProjectService.ProjectService (
            IRepository< Project > projectRepository)
```

Instantiates the ProjectService utilizing the provided storage repository.

Parameters

projectRepository	The instantiated storage repository mechanism (dependency).

Definition at line 24 of file ProjectService.java.

```
00024                                                                {
00025         this.projectRepository = projectRepository;
00026     }
```

8.11.3 Member Function Documentation

8.11.3.1 addTaskToProject()

```
void com.projectmanagement.lib.services.ProjectService.addTaskToProject (
            String projectId,
            Task task)
```

Appends a Task to a persistent Project based on the Project's id.

Parameters

projectId	Valid identifier representing the target Project.
task	System-ready Task object to attach to the Project.

Exceptions

IllegalArgumentException	If the given projectId cannot be found in the current store.

Definition at line 50 of file ProjectService.java.

```
00050                                                                   {
00051         if (projectId == null || task == null) {
00052             throw new IllegalArgumentException("Target Project identifier and Task components must be
      provided.");
00053         }
00054
00055         Project existingProject = projectRepository.findById(projectId);
00056         if (existingProject == null) {
00057              throw new IllegalArgumentException("Targeted Project does not exist.");
00058         }
00059
00060         existingProject.addTask(task);
00061         projectRepository.update(existingProject); // Merges the modified relation structure back to
      the Repo
00062     }
```

References com.projectmanagement.lib.models.Project.addTask().

8.11.3.2 createProject()

```
void com.projectmanagement.lib.services.ProjectService.createProject (
            String id,
            String name,
            String description)
```

Establishes a new Project and commits it to the designated storage backend.

Parameters

id	System-unique identifier for the Project.
name	Name branding the Project.
description	Full textual breakdown summarizing the Project's scope.

Exceptions

IllegalArgumentException	If core identifying attributes are omitted.

Definition at line 35 of file ProjectService.java.

```
00035                                                                   {
00036         if (id == null || name == null || name.isEmpty() || description == null) {
00037             throw new IllegalArgumentException("Project initialization values cannot be void or
      blank.");
00038         }
00039
00040         Project newProject = new Project(id, name, description);
00041         projectRepository.save(newProject);
00042     }
```

8.11.3.3 getAllProjects()

```
List< Project > com.projectmanagement.lib.services.ProjectService.getAllProjects ()
```

Retrieves all registered projects from the repository.

Returns

List containing all Project instances.

Definition at line 68 of file ProjectService.java.

```
00068                                    {
00069         return projectRepository.findAll();
00070     }
```

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/services/ProjectService.java

8.12 com.projectmanagement.lib.services.ReportService Class Reference

Public Member Functions

- ReportService (ProjectService projectService, TaskService taskService, UserService userService)
- void generateGeneralSystemReport ()
- void generateTaskStatusReport ()

8.12.1 Detailed Description

Definition at line 10 of file ReportService.java.

8.12.2 Constructor & Destructor Documentation

8.12.2.1 ReportService()

```
com.projectmanagement.lib.services.ReportService.ReportService (
            ProjectService projectService,
            TaskService taskService,
            UserService userService)
```

Definition at line 16 of file ReportService.java.

```
00016
      {
00017         this.projectService = projectService;
00018         this.taskService = taskService;
00019         this.userService = userService;
00020     }
```

8.12.3 Member Function Documentation

8.12.3.1 generateGeneralSystemReport()

```
void com.projectmanagement.lib.services.ReportService.generateGeneralSystemReport ()
```

Definition at line 22 of file ReportService.java.

```
00022                                                {
00023         System.out.println("\n=====================================");
00024         System.out.println("       GENERAL SYSTEM REPORT         ");
00025         System.out.println("=====================================");
00026
00027         List<Project> projects = projectService.getAllProjects();
00028         List<Task> tasks = taskService.getAllTasks();
00029         List<User> users = userService.getAllUsers();
00030
00031         System.out.println("Total Registered Users  : " + (users != null ? users.size() : 0));
00032         System.out.println("Total Managed Projects  : " + (projects != null ? projects.size() : 0));
00033         System.out.println("Total Tracked Tasks     : " + (tasks != null ? tasks.size() : 0));
00034
00035         if (projects != null && !projects.isEmpty()) {
00036             System.out.println("\n--- Projects Breakdown ---");
00037             for (Project project : projects) {
00038                 System.out.println(" [Project] " + project.getName() + " (Contains " +
      project.getTasks().size() + " tasks)");
00039             }
00040         }
00041         System.out.println("=====================================\n");
00042     }
```

8.12.3.2 generateTaskStatusReport()

```
void com.projectmanagement.lib.services.ReportService.generateTaskStatusReport ()
```

Definition at line 44 of file ReportService.java.

```
00044                                                {
00045         System.out.println("\n=====================================");
00046         System.out.println("         TASK STATUS REPORT          ");
00047         System.out.println("=====================================");
00048
00049         List<Task> tasks = taskService.getAllTasks();
00050         if (tasks == null || tasks.isEmpty()) {
00051             System.out.println("No tasks found in the system.");
00052             System.out.println("=====================================\n");
00053             return;
00054         }
00055
00056         long todo = tasks.stream().filter(t -> t.getStatus() == TaskStatus.TODO).count();
00057         long inProgress = tasks.stream().filter(t -> t.getStatus() == TaskStatus.IN_PROGRESS).count();
00058         long done = tasks.stream().filter(t -> t.getStatus() == TaskStatus.DONE).count();
00059
00060         System.out.println("Total Tasks     : " + tasks.size());
00061         System.out.println("TODO            : " + todo);
00062         System.out.println("IN_PROGRESS     : " + inProgress);
00063         System.out.println("DONE            : " + done);
00064         System.out.println("=====================================\n");
00065     }
```

References com.projectmanagement.lib.models.TaskStatus.DONE, com.projectmanagement.lib.models.TaskStatus.IN_PROGRESS, and com.projectmanagement.lib.models.TaskStatus.TODO.

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/services/ReportService.java

8.13 com.projectmanagement.lib.storage.RepositoryFactory Class Reference

Public Member Functions

- RepositoryFactory (StorageConfig config)
- IRepository< User > getUserRepository ()
- IRepository< Project > getProjectRepository ()
- IRepository< Task > getTaskRepository ()

8.13.1 Detailed Description

Factory class to resolve and retrieve specific storage repositories.

Examines the current StorageConfig and supplies the correct implementation (Binary, SQLite, MySQL) for User, Project, or Task. Applies the Factory Design Pattern.

Definition at line 11 of file RepositoryFactory.java.

8.13.2 Constructor & Destructor Documentation

8.13.2.1 RepositoryFactory()

```
com.projectmanagement.lib.storage.RepositoryFactory.RepositoryFactory (
            StorageConfig config)
```

Constructor for the RepositoryFactory.

Parameters

config	The active storage configuration to reference during instantiation.

Definition at line 22 of file RepositoryFactory.java.

```
00022                                                   {
00023         this.config = config;
00024     }
```

8.13.3 Member Function Documentation

8.13.3.1 getProjectRepository()

```
IRepository< Project > com.projectmanagement.lib.storage.RepositoryFactory.getProjectRepository
()
```

Generates the matched IRepository for Project domains.

Returns

The correctly typed Project repository.

Definition at line 45 of file RepositoryFactory.java.

```
00045                                                    {
00046         if (config.getActiveStorageType() == StorageType.BINARY_FILE) {
00047             return new BinaryProjectRepository();
00048         } else if (config.getActiveStorageType() == StorageType.SQLITE) {
00049             return new SQLiteProjectRepository();
00050         } else if (config.getActiveStorageType() == StorageType.MYSQL) {
00051             return new MySQLProjectRepository();
00052         }
00053         throw new IllegalStateException("Unsupported storage type for Projects");
00054     }
```

References com.projectmanagement.lib.storage.StorageType.BINARY_FILE, com.projectmanagement.lib.storage.StorageType.MYSQ and com.projectmanagement.lib.storage.StorageType.SQLITE.

8.13.3.2 getTaskRepository()

```
IRepository< Task > com.projectmanagement.lib.storage.RepositoryFactory.getTaskRepository ()
```

Generates the matched IRepository for Task domains.

Returns

The correctly typed Task repository.

Definition at line 60 of file RepositoryFactory.java.

```
00060                                                    {
00061         if (config.getActiveStorageType() == StorageType.BINARY_FILE) {
00062             return new BinaryTaskRepository();
00063         } else if (config.getActiveStorageType() == StorageType.SQLITE) {
00064             return new SQLiteTaskRepository();
00065         } else if (config.getActiveStorageType() == StorageType.MYSQL) {
00066             return new MySQLTaskRepository();
00067         }
00068         throw new IllegalStateException("Unsupported storage type for Tasks");
00069     }
```

References com.projectmanagement.lib.storage.StorageType.BINARY_FILE, com.projectmanagement.lib.storage.StorageType.MYSQ and com.projectmanagement.lib.storage.StorageType.SQLITE.

8.13.3.3 getUserRepository()

```
IRepository< User > com.projectmanagement.lib.storage.RepositoryFactory.getUserRepository ()
```

Generates the matched IRepository for User domains.

Returns

The correctly typed User repository.

Definition at line 30 of file RepositoryFactory.java.

```
00030                                                    {
00031         if (config.getActiveStorageType() == StorageType.BINARY_FILE) {
00032             return new BinaryUserRepository();
00033         } else if (config.getActiveStorageType() == StorageType.SQLITE) {
00034             return new SQLiteUserRepository();
00035         } else if (config.getActiveStorageType() == StorageType.MYSQL) {
00036             return new MySQLUserRepository();
00037         }
00038         throw new IllegalStateException("Unsupported storage type for Users");
00039     }
```

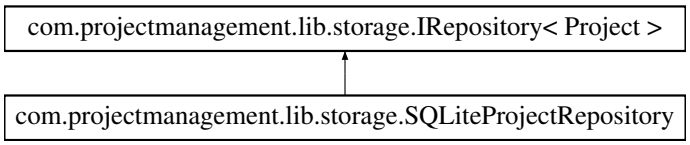
References com.projectmanagement.lib.storage.StorageType.BINARY_FILE, com.projectmanagement.lib.storage.StorageType.MYSQ and com.projectmanagement.lib.storage.StorageType.SQLITE.

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/storage/RepositoryFactory.java

8.14 com.projectmanagement.lib.storage.SQLiteProjectRepository Class Reference

Inheritance diagram for com.projectmanagement.lib.storage.SQLiteProjectRepository:



Public Member Functions

- SQLiteProjectRepository ()
- void save (Project entity)
- Project findById (String id)
- List< Project > findAll ()
- void update (Project entity)
- void delete (String id)

Public Member Functions inherited from com.projectmanagement.lib.storage.IRepository< Project >

- void save (T entity)
- T findById (String id)
- List< T > findAll ()
- void update (T entity)
- void delete (String id)

Protected Member Functions

- Connection getConnection () throws SQLException

8.14.1 Detailed Description

SQLite implementation of the Project repository.

Implements IRepository to persist Project entities to a local SQLite database.

Definition at line 12 of file SQLiteProjectRepository.java.

8.14.2 Constructor & Destructor Documentation

8.14.2.1 SQLiteProjectRepository()

```
com.projectmanagement.lib.storage.SQLiteProjectRepository.SQLiteProjectRepository ()
```

Constructor for SQLiteProjectRepository.

Definition at line 22 of file SQLiteProjectRepository.java.

```
00022                                   {
00023         createTableIfNotExists();
00024     }
```

8.14.3 Member Function Documentation

8.14.3.1 delete()

```
void com.projectmanagement.lib.storage.SQLiteProjectRepository.delete (
            String id)
```

Eradicates a Project from the SQLite database stream by ID.

Parameters

id	Targeted Project identifier.

Definition at line 181 of file SQLiteProjectRepository.java.

```
00181                                   {
00182         String sql = "DELETE FROM projects WHERE id = ?";
00183         try (Connection conn = getConnection();
00184              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00185             pstmt.setString(1, id);
00186             pstmt.executeUpdate();
00187
00188             String delSql = "DELETE FROM project_tasks WHERE project_id = ?";
00189             try (PreparedStatement delStmt = conn.prepareStatement(delSql)) {
00190                 delStmt.setString(1, id);
00191                 delStmt.executeUpdate();
00192             }
00193         } catch (SQLException e) {
00194             System.err.println("Error deleting project: " + e.getMessage());
00195         }
00196     }
```

References getConnection().

8.14.3.2 findAll()

```
List< Project > com.projectmanagement.lib.storage.SQLiteProjectRepository.findAll ()
```

Retrieves the entirety of Project records from the SQLite db.

Returns

Complete list of Project occurrences.

Definition at line 135 of file SQLiteProjectRepository.java.

```
00135                                   {
00136         List<Project> projects = new ArrayList<>();
00137         String sql = "SELECT * FROM projects";
00138         try (Connection conn = getConnection();
00139              Statement stmt = conn.createStatement();
00140              ResultSet rs = stmt.executeQuery(sql)) {
00141             while (rs.next()) {
00142                 Project project = new Project(
00143                     rs.getString("id"),
00144                     rs.getString("name"),
00145                     rs.getString("description")
00146                 );
00147                 loadProjectTasks(conn, project);
00148                 projects.add(project);
00149             }
00150         } catch (SQLException e) {
00151             System.err.println("Error finding all projects: " + e.getMessage());
00152         }
00153         return projects;
00154     }
```

References getConnection().

8.14.3.3 findById()

```
Project com.projectmanagement.lib.storage.SQLiteProjectRepository.findById (
            String id)
```

Looks up a Project from the SQLite db based on ID.

Parameters

id	The sought-after Project ID.

Returns

The distinct Project, or null if undiscovered.

Definition at line 94 of file SQLiteProjectRepository.java.

```
00094                                       {
00095         String sql = "SELECT * FROM projects WHERE id = ?";
00096         try (Connection conn = getConnection();
00097              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00098             pstmt.setString(1, id);
00099             ResultSet rs = pstmt.executeQuery();
00100             if (rs.next()) {
00101                 Project project = new Project(
00102                     rs.getString("id"),
00103                     rs.getString("name"),
00104                     rs.getString("description")
00105                 );
00106                 loadProjectTasks(conn, project);
00107                 return project;
00108             }
00109         } catch (SQLException e) {
00110             System.err.println("Error finding project by id: " + e.getMessage());
00111         }
00112         return null;
00113     }
```

References getConnection().

8.14.3.4 getConnection()

```
Connection com.projectmanagement.lib.storage.SQLiteProjectRepository.getConnection () throws
SQLException  [protected]
```

Definition at line 198 of file SQLiteProjectRepository.java.

```
00198 { return DriverManager.getConnection(URL); }
```

Referenced by delete(), findAll(), findById(), save(), and update().

8.14.3.5 save()

```
void com.projectmanagement.lib.storage.SQLiteProjectRepository.save (
            Project entity)
```

Inserts a new Project record into the SQLite db.

Parameters

entity	The Project instance to be added.

Definition at line 54 of file SQLiteProjectRepository.java.

```
00054                                   {
00055         String sql = "INSERT INTO projects(id, name, description) VALUES(?, ?, ?)";
00056         try (Connection conn = getConnection();
00057              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00058             pstmt.setString(1, entity.getId());
00059             pstmt.setString(2, entity.getName());
00060             pstmt.setString(3, entity.getDescription());
00061             pstmt.executeUpdate();
00062
00063             saveProjectTasks(conn, entity);
00064         } catch (SQLException e) {
00065             System.err.println("Error creating project: " + e.getMessage());
00066         }
00067     }
```

References getConnection(), com.projectmanagement.lib.models.Project.getDescription(), com.projectmanagement.lib.models.BaseE and com.projectmanagement.lib.models.Project.getName().

8.14.3.6 update()

```
void com.projectmanagement.lib.storage.SQLiteProjectRepository.update (
            Project entity)
```

Applies modifications to a documented Project in the SQLite db.

Parameters

entity	Project entity carrying current revisions.

Definition at line 161 of file SQLiteProjectRepository.java.

```
00161                                   {
00162         String sql = "UPDATE projects SET name = ?, description = ? WHERE id = ?";
00163         try (Connection conn = getConnection();
00164              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00165             pstmt.setString(1, entity.getName());
00166             pstmt.setString(2, entity.getDescription());
00167             pstmt.setString(3, entity.getId());
00168             pstmt.executeUpdate();
00169
00170             saveProjectTasks(conn, entity);
00171         } catch (SQLException e) {
00172             System.err.println("Error updating project: " + e.getMessage());
00173         }
00174     }
```

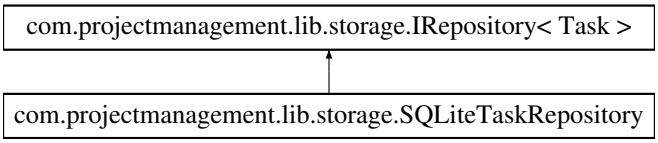
References getConnection(), com.projectmanagement.lib.models.Project.getDescription(), com.projectmanagement.lib.models.BaseE and com.projectmanagement.lib.models.Project.getName().

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/storage/SQLiteProjectRepository.java

8.15 com.projectmanagement.lib.storage.SQLiteTaskRepository Class Reference

Inheritance diagram for com.projectmanagement.lib.storage.SQLiteTaskRepository:



Public Member Functions

- SQLiteTaskRepository ()
- void save (Task entity)
- Task findById (String id)
- List< Task > findAll ()
- void update (Task entity)
- void delete (String id)

Public Member Functions inherited from com.projectmanagement.lib.storage.IRepository< Task >

- void save (T entity)
- T findById (String id)
- List< T > findAll ()
- void update (T entity)
- void delete (String id)

Protected Member Functions

- Connection getConnection () throws SQLException

8.15.1 Detailed Description

SQLite implementation of the Task repository.

Implements IRepository to persist Task entities securely within an SQLite local db.

Definition at line 14 of file SQLiteTaskRepository.java.

8.15.2 Constructor & Destructor Documentation

8.15.2.1 SQLiteTaskRepository()

```
com.projectmanagement.lib.storage.SQLiteTaskRepository.SQLiteTaskRepository ()
```

Constructor for SQLiteTaskRepository.

Definition at line 24 of file SQLiteTaskRepository.java.

```
00024                                   {
00025         createTableIfNotExists();
00026     }
```

8.15.3 Member Function Documentation

8.15.3.1 delete()

```
void com.projectmanagement.lib.storage.SQLiteTaskRepository.delete (
            String id)
```

Withdraws a given Task from the associated SQLite db using ID.

Parameters

id	The corresponding Task id string.

Definition at line 155 of file SQLiteTaskRepository.java.

```
00155                                   {
00156         String sql = "DELETE FROM tasks WHERE id = ?";
00157         try (Connection conn = getConnection();
00158              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00159             pstmt.setString(1, id);
00160             pstmt.executeUpdate();
00161         } catch (SQLException e) {
00162             System.err.println("Error deleting task: " + e.getMessage());
00163         }
00164     }
```

References getConnection().

8.15.3.2 findAll()

```
List< Task > com.projectmanagement.lib.storage.SQLiteTaskRepository.findAll ()
```

Discovers and yields all Task forms positioned in the SQLite db.

Returns

Exhaustive list collection of Tasks.

Definition at line 104 of file SQLiteTaskRepository.java.

```
00104                                   {
00105         List<Task> tasks = new ArrayList<>();
00106         String sql = "SELECT * FROM tasks";
00107         try (Connection conn = getConnection();
00108              Statement stmt = conn.createStatement();
00109              ResultSet rs = stmt.executeQuery(sql)) {
00110             SQLiteUserRepository userRepo = new SQLiteUserRepository();
00111             while (rs.next()) {
00112                 Task task = new Task(
00113                     rs.getString("id"),
00114                     rs.getString("title"),
00115                     rs.getString("description")
00116                 );
00117                 task.setStatus(TaskStatus.valueOf(rs.getString("status")));
00118                 String userId = rs.getString("assigned_user_id");
00119                 if (userId != null) {
00120                     task.setAssignedUser(userRepo.findById(userId));
00121                 }
00122                 tasks.add(task);
00123             }
00124         } catch (SQLException e) {
00125             System.err.println("Error finding all tasks: " + e.getMessage());
00126         }
00127         return tasks;
00128     }
```

References com.projectmanagement.lib.storage.SQLiteUserRepository.findById(), getConnection(), com.projectmanagement.lib.mod
and com.projectmanagement.lib.models.Task.setStatus().

8.15.3.3 findById()

```
Task com.projectmanagement.lib.storage.SQLiteTaskRepository.findById (
            String id)
```

Reads a specific Task from the SQLite db using its ID.

Parameters

id	Tracking index of the desired Task.

Returns

Single Task reference, or null when undiscovered.

Definition at line 73 of file SQLiteTaskRepository.java.

```
00073                                   {
00074         String sql = "SELECT * FROM tasks WHERE id = ?";
00075         try (Connection conn = getConnection();
00076              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00077             pstmt.setString(1, id);
00078             ResultSet rs = pstmt.executeQuery();
00079             if (rs.next()) {
00080                 Task task = new Task(
00081                     rs.getString("id"),
00082                     rs.getString("title"),
00083                     rs.getString("description")
00084                 );
00085                 task.setStatus(TaskStatus.valueOf(rs.getString("status")));
00086                 String userId = rs.getString("assigned_user_id");
00087                 if (userId != null) {
00088                     User assignee = new SQLiteUserRepository().findById(userId);
00089                     task.setAssignedUser(assignee);
00090                 }
00091                 return task;
00092             }
00093         } catch (SQLException e) {
00094             System.err.println("Error finding task by id: " + e.getMessage());
00095         }
00096         return null;
00097     }
```

References com.projectmanagement.lib.storage.SQLiteUserRepository.findById(), getConnection(), com.projectmanagement.lib.mod
and com.projectmanagement.lib.models.Task.setStatus().

8.15.3.4 getConnection()

```
Connection com.projectmanagement.lib.storage.SQLiteTaskRepository.getConnection () throws
SQLException [protected]
```

Definition at line 166 of file SQLiteTaskRepository.java.

```
00166 { return DriverManager.getConnection(URL); }
```

Referenced by delete(), findAll(), findById(), save(), and update().

8.15.3.5 save()

```
void com.projectmanagement.lib.storage.SQLiteTaskRepository.save (
            Task entity)
```

Initiates a Task entity registry within the SQLite db.

Parameters

entity	The Task implementation object.

Definition at line 52 of file SQLiteTaskRepository.java.

```
00052                                   {
00053         String sql = "INSERT INTO tasks(id, title, description, status, assigned_user_id) VALUES(?, ?,
      ?, ?, ?)";
00054         try (Connection conn = getConnection();
00055              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00056             pstmt.setString(1, entity.getId());
00057             pstmt.setString(2, entity.getTitle());
00058             pstmt.setString(3, entity.getDescription());
00059             pstmt.setString(4, entity.getStatus().name());
00060             pstmt.setString(5, entity.getAssignedUser() != null ? entity.getAssignedUser().getId() :
      null);
00061             pstmt.executeUpdate();
00062         } catch (SQLException e) {
00063             System.err.println("Error creating task: " + e.getMessage());
00064         }
00065     }
```

References com.projectmanagement.lib.models.Task.getAssignedUser(), getConnection(), com.projectmanagement.lib.models.Task.g
com.projectmanagement.lib.models.BaseEntity.getId(), com.projectmanagement.lib.models.Task.getStatus(), and com.projectmanagement.lib.models.Task.getTitle().

8.15.3.6 update()

```
void com.projectmanagement.lib.storage.SQLiteTaskRepository.update (
            Task entity)
```

Re-registers an associated Task inside SQLite with incoming tweaks.

Parameters

entity	Modified Task form.

Definition at line 135 of file SQLiteTaskRepository.java.

```
00135                                   {
00136         String sql = "UPDATE tasks SET title = ?, description = ?, status = ?, assigned_user_id = ?
      WHERE id = ?";
00137         try (Connection conn = getConnection();
00138              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00139             pstmt.setString(1, entity.getTitle());
00140             pstmt.setString(2, entity.getDescription());
00141             pstmt.setString(3, entity.getStatus().name());
00142             pstmt.setString(4, entity.getAssignedUser() != null ? entity.getAssignedUser().getId() :
      null);
00143             pstmt.setString(5, entity.getId());
00144             pstmt.executeUpdate();
00145         } catch (SQLException e) {
00146             System.err.println("Error updating task: " + e.getMessage());
00147         }
00148     }
```

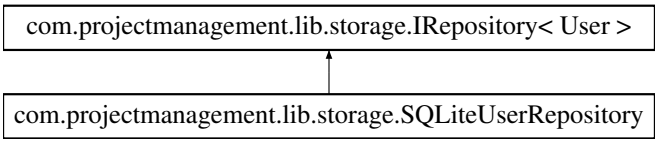
References com.projectmanagement.lib.models.Task.getAssignedUser(), getConnection(), com.projectmanagement.lib.models.Task.g
com.projectmanagement.lib.models.BaseEntity.getId(), com.projectmanagement.lib.models.Task.getStatus(), and com.projectmanagement.lib.models.Task.getTitle().

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/storage/SQLiteTaskRepository.java

8.16 com.projectmanagement.lib.storage.SQLiteUserRepository Class Reference

Inheritance diagram for com.projectmanagement.lib.storage.SQLiteUserRepository:



Public Member Functions

- SQLiteUserRepository ()
- void save (User entity)
- User findById (String id)
- List< User > findAll ()
- void update (User entity)
- void delete (String id)

Public Member Functions inherited from com.projectmanagement.lib.storage.IRepository< User >

- void save (T entity)
- T findById (String id)
- List< T > findAll ()
- void update (T entity)
- void delete (String id)

Protected Member Functions

- Connection getConnection () throws SQLException

8.16.1 Detailed Description

SQLite implementation of the User repository.

Implements IRepository to persist User data to a local SQLite database.

Definition at line 12 of file SQLiteUserRepository.java.

8.16.2 Constructor & Destructor Documentation

8.16.2.1 SQLiteUserRepository()

```
com.projectmanagement.lib.storage.SQLiteUserRepository.SQLiteUserRepository ()
```

Constructor for SQLiteUserRepository.

Ensures that the users table exists when the repository is created.

Definition at line 23 of file SQLiteUserRepository.java.

```
00023                                   {
00024         createTableIfNotExists();
00025     }
```

8.16.3 Member Function Documentation

8.16.3.1 delete()

```
void com.projectmanagement.lib.storage.SQLiteUserRepository.delete (
            String id)
```

Evicts a User from the SQLite database by ID.

Parameters

id	The ID of the User to remove.

Definition at line 139 of file SQLiteUserRepository.java.

```
00139                                   {
00140         String sql = "DELETE FROM users WHERE id = ?";
00141         try (Connection conn = getConnection();
00142              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00143             pstmt.setString(1, id);
00144             pstmt.executeUpdate();
00145         } catch (SQLException e) {
00146             System.err.println("Error deleting user: " + e.getMessage());
00147         }
00148     }
```

References getConnection().

8.16.3.2 findAll()

```
List< User > com.projectmanagement.lib.storage.SQLiteUserRepository.findAll ()
```

Retrieves all Users stored in the SQLite db.

Returns

A list of all User entities.

Definition at line 95 of file SQLiteUserRepository.java.

```
00095                                   {
00096         List<User> users = new ArrayList<>();
00097         String sql = "SELECT * FROM users";
00098         try (Connection conn = getConnection();
00099              Statement stmt = conn.createStatement();
00100              ResultSet rs = stmt.executeQuery(sql)) {
00101             while (rs.next()) {
00102                 users.add(new User(
00103                     rs.getString("id"),
00104                     rs.getString("username"),
00105                     rs.getString("email"),
00106                     rs.getString("password")
00107                 ));
00108             }
00109         } catch (SQLException e) {
00110             System.err.println("Error finding all users: " + e.getMessage());
00111         }
00112         return users;
00113     }
```

References getConnection().

8.16.3.3 findById()

```
User com.projectmanagement.lib.storage.SQLiteUserRepository.findById (
            String id)
```

Queries for a User by identifier from the SQLite db.

Parameters

id	The unique identifier.

Returns

The matched User, or null if missing.

Definition at line 70 of file SQLiteUserRepository.java.

```
00070                                   {
00071         String sql = "SELECT * FROM users WHERE id = ?";
00072         try (Connection conn = getConnection();
00073              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00074             pstmt.setString(1, id);
00075             ResultSet rs = pstmt.executeQuery();
00076             if (rs.next()) {
00077                 return new User(
00078                     rs.getString("id"),
00079                     rs.getString("username"),
00080                     rs.getString("email"),
00081                     rs.getString("password")
00082                 );
00083             }
00084         } catch (SQLException e) {
00085             System.err.println("Error finding user by id: " + e.getMessage());
00086         }
00087         return null;
00088     }
```

References getConnection().

Referenced by com.projectmanagement.lib.storage.SQLiteTaskRepository.findAll(), and com.projectmanagement.lib.storage.SQLiteTa

8.16.3.4 getConnection()

```
Connection com.projectmanagement.lib.storage.SQLiteUserRepository.getConnection () throws
SQLException [protected]
```

Definition at line 150 of file SQLiteUserRepository.java.

```
00150 { return DriverManager.getConnection(URL); }
```

Referenced by delete(), findAll(), findById(), save(), and update().

8.16.3.5 save()

```
void com.projectmanagement.lib.storage.SQLiteUserRepository.save (
            User entity)
```

Inserts a new User into the SQLite db.

Parameters

entity	The User object to insert.

Definition at line 50 of file SQLiteUserRepository.java.

```
00050                                   {
00051         String sql = "INSERT INTO users(id, username, email, password) VALUES(?, ?, ?, ?)";
00052         try (Connection conn = getConnection();
00053              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00054             pstmt.setString(1, entity.getId());
00055             pstmt.setString(2, entity.getUsername());
00056             pstmt.setString(3, entity.getEmail());
00057             pstmt.setString(4, entity.getPassword());
00058             pstmt.executeUpdate();
00059         } catch (SQLException e) {
00060             System.err.println("Error creating user: " + e.getMessage());
00061         }
00062     }
```

References getConnection(), com.projectmanagement.lib.models.User.getEmail(), com.projectmanagement.lib.models.BaseEntity.get
com.projectmanagement.lib.models.User.getPassword(), and com.projectmanagement.lib.models.User.getUsername().

8.16.3.6 update()

```
void com.projectmanagement.lib.storage.SQLiteUserRepository.update (
            User entity)
```

Updates an existing User's attributes in the SQLite db.

Parameters

entity	The User instance containing modified data.

Definition at line 120 of file SQLiteUserRepository.java.

```
00120                                   {
00121         String sql = "UPDATE users SET username = ?, email = ?, password = ? WHERE id = ?";
00122         try (Connection conn = getConnection();
00123              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00124             pstmt.setString(1, entity.getUsername());
00125             pstmt.setString(2, entity.getEmail());
00126             pstmt.setString(3, entity.getPassword());
00127             pstmt.setString(4, entity.getId());
00128             pstmt.executeUpdate();
00129         } catch (SQLException e) {
00130             System.err.println("Error updating user: " + e.getMessage());
00131         }
00132     }
```

References getConnection(), com.projectmanagement.lib.models.User.getEmail(), com.projectmanagement.lib.models.BaseEntity.get
com.projectmanagement.lib.models.User.getPassword(), and com.projectmanagement.lib.models.User.getUsername().

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/storage/SQLiteUserRepository.java

8.17 com.projectmanagement.lib.storage.StorageConfig Class Reference

Public Member Functions

- StorageConfig (StorageType defaultStorage)
- StorageType getActiveStorageType ()
- void setActiveStorageType (StorageType activeStorageType)

8.17.1 Detailed Description

Manages the application's current active storage type configuration.

Configures whether the application will use Binary file, SQLite, or MySQL.

Definition at line 7 of file StorageConfig.java.

8.17.2 Constructor & Destructor Documentation

8.17.2.1 StorageConfig()

```
com.projectmanagement.lib.storage.StorageConfig.StorageConfig (
            StorageType defaultStorage)
```

Constructor for the storage configuration.

Parameters

defaultStorage	The initial storage backend type to use.

Definition at line 18 of file StorageConfig.java.

```
00018                                                   {
00019         this.activeStorageType = defaultStorage;
00020     }
```

8.17.3 Member Function Documentation

8.17.3.1 getActiveStorageType()

```
StorageType com.projectmanagement.lib.storage.StorageConfig.getActiveStorageType ()
```

Gets the active storage backend type.

Returns

The active StorageType.

Definition at line 26 of file StorageConfig.java.

```
00026                                           {
00027         return activeStorageType;
00028     }
```

8.17.3.2 setActiveStorageType()

```
void com.projectmanagement.lib.storage.StorageConfig.setActiveStorageType (
            StorageType activeStorageType)
```

Sets the active storage backend type.

Parameters

activeStorageType	The new StorageType to set.

Definition at line 34 of file StorageConfig.java.

```
00034                                                                   {
00035         this.activeStorageType = activeStorageType;
00036     }
```

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/storage/StorageConfig.java

8.18 com.projectmanagement.lib.storage.StorageType Enum Reference

Public Attributes

- BINARY_FILE
- SQLITE
- MYSQL

8.18.1 Detailed Description

Enum representing the supported storage backend types.

Definition at line 6 of file StorageType.java.

8.18.2 Member Data Documentation

8.18.2.1 BINARY_FILE

```
com.projectmanagement.lib.storage.StorageType.BINARY_FILE
```

Indicates that data will be stored using Binary File I/O.

Definition at line 10 of file StorageType.java.

Referenced by com.projectmanagement.lib.storage.RepositoryFactory.getProjectRepository(), com.projectmanagement.lib.storage.Re
and com.projectmanagement.lib.storage.RepositoryFactory.getUserRepository().

8.18.2.2 MYSQL

`com.projectmanagement.lib.storage.StorageType.MYSQL`

Indicates that data will be stored using a MySQL database.

Definition at line 20 of file StorageType.java.

Referenced by com.projectmanagement.lib.storage.RepositoryFactory.getProjectRepository(), com.projectmanagement.lib.storage.Re
and com.projectmanagement.lib.storage.RepositoryFactory.getUserRepository().

8.18.2.3 SQLITE

`com.projectmanagement.lib.storage.StorageType.SQLITE`

Indicates that data will be stored using an SQLite database.

Definition at line 15 of file StorageType.java.

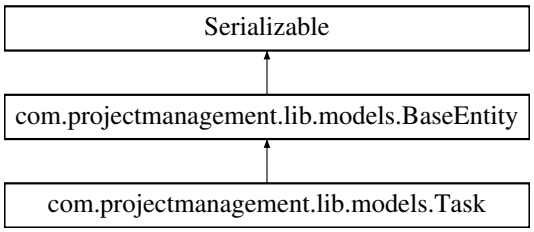
Referenced by com.projectmanagement.lib.storage.RepositoryFactory.getProjectRepository(), com.projectmanagement.lib.storage.Re
and com.projectmanagement.lib.storage.RepositoryFactory.getUserRepository().

The documentation for this enum was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/storage/StorageType.java

8.19 com.projectmanagement.lib.models.Task Class Reference

Inheritance diagram for com.projectmanagement.lib.models.Task:



Public Member Functions

- Task ()
- Task (String id, String title, String description)
- String getTitle ()
- void setTitle (String title)
- String getDescription ()
- void setDescription (String description)
- TaskStatus getStatus ()
- void setStatus (TaskStatus status)
- User getAssignedUser ()
- void setAssignedUser (User assignedUser)
- void displayDetails ()

Public Member Functions inherited from com.projectmanagement.lib.models.BaseEntity

- BaseEntity ()
- BaseEntity (String id)
- String getId ()
- void setId (String id)
- LocalDateTime getCreatedDate ()
- void setCreatedDate (LocalDateTime createdDate)

8.19.1 Detailed Description

Represents a task within a project.

Inherits from BaseEntity. Encapsulates task data such as title, status, and the assigned user.

Definition at line 7 of file Task.java.

8.19.2 Constructor & Destructor Documentation

8.19.2.1 Task() [1/2]

```
com.projectmanagement.lib.models.Task.Task ()
```

Default constructor for Task.

Sets the default status to TODO.

Definition at line 32 of file Task.java.

```
00032                   {
00033         super();
00034         this.status = TaskStatus.TODO;
00035     }
```

References com.projectmanagement.lib.models.TaskStatus.TODO.

8.19.2.2 Task() [2/2]

```
com.projectmanagement.lib.models.Task.Task (
            String id,
            String title,
            String description)
```

Parameterized constructor for Task.

Parameters

id	The unique identifier of the task.
title	The title of the task.
description	The task description.

Definition at line 43 of file Task.java.

```
00043                                                       {
00044         super(id);
00045         this.title = title;
00046         this.description = description;
00047         this.status = TaskStatus.TODO;
00048     }
```

References com.projectmanagement.lib.models.TaskStatus.TODO.

8.19.3 Member Function Documentation

8.19.3.1 displayDetails()

```
void com.projectmanagement.lib.models.Task.displayDetails ()
```

Displays the details of the task.

Implementation of the abstract method from BaseEntity.

Reimplemented from com.projectmanagement.lib.models.BaseEntity.

Definition at line 119 of file Task.java.

```
00119                                   {
00120         System.out.println("Task ID: " + getId() + ", Title: " + title);
00121         System.out.println("Status: " + status);
00122         if (assignedUser != null) {
00123             System.out.println("Assigned To: " + assignedUser.getUsername());
00124         } else {
00125             System.out.println("Assigned To: Unassigned");
00126         }
00127     }
```

References com.projectmanagement.lib.models.BaseEntity.getId().

8.19.3.2 getAssignedUser()

```
User com.projectmanagement.lib.models.Task.getAssignedUser ()
```

Gets the user assigned to the task.

Returns

The assigned user, or null if unassigned.

Definition at line 102 of file Task.java.

```
00102                                   {
00103         return assignedUser;
00104     }
```

Referenced by com.projectmanagement.lib.storage.MySQLTaskRepository.save(), com.projectmanagement.lib.storage.SQLiteTaskRe
com.projectmanagement.lib.storage.MySQLTaskRepository.update(), and com.projectmanagement.lib.storage.SQLiteTaskRepository

8.19.3.3 getDescription()

```
String com.projectmanagement.lib.models.Task.getDescription ()
```

Gets the description of the task.

Returns

The task description.

Definition at line 70 of file Task.java.

```
00070                                   {
00071         return description;
00072     }
```

Referenced by com.projectmanagement.lib.storage.MySQLTaskRepository.save(), com.projectmanagement.lib.storage.SQLiteTaskRe
com.projectmanagement.lib.storage.MySQLTaskRepository.update(), and com.projectmanagement.lib.storage.SQLiteTaskRepository

8.19.3.4 getStatus()

```
TaskStatus com.projectmanagement.lib.models.Task.getStatus ()
```

Gets the current status of the task.

Returns

The task status.

Definition at line 86 of file Task.java.

```
00086                                   {
00087         return status;
00088     }
```

Referenced by com.projectmanagement.lib.storage.MySQLTaskRepository.save(), com.projectmanagement.lib.storage.SQLiteTaskRe
com.projectmanagement.lib.storage.MySQLTaskRepository.update(), and com.projectmanagement.lib.storage.SQLiteTaskRepository

8.19.3.5 getTitle()

```
String com.projectmanagement.lib.models.Task.getTitle ()
```

Gets the title of the task.

Returns

The task title.

Definition at line 54 of file Task.java.

```
00054                                 {
00055         return title;
00056     }
```

Referenced by com.projectmanagement.lib.storage.MySQLTaskRepository.save(), com.projectmanagement.lib.storage.SQLiteTaskRe
com.projectmanagement.lib.storage.MySQLTaskRepository.update(), and com.projectmanagement.lib.storage.SQLiteTaskRepository

8.19.3.6 setAssignedUser()

```
void com.projectmanagement.lib.models.Task.setAssignedUser (
            User assignedUser)
```

Assigns a user to the task.

Parameters

assignedUser	The user to be assigned to this task.

Definition at line 110 of file Task.java.

```
00110                                                   {
00111         this.assignedUser = assignedUser;
00112     }
```

Referenced by com.projectmanagement.lib.services.TaskService.assignTaskToUser(), com.projectmanagement.lib.storage.MySQLTa
com.projectmanagement.lib.storage.SQLiteTaskRepository.findAll(), com.projectmanagement.lib.storage.MySQLTaskRepository.findE
and com.projectmanagement.lib.storage.SQLiteTaskRepository.findById().

8.19.3.7 setDescription()

```
void com.projectmanagement.lib.models.Task.setDescription (
            String description)
```

Sets the description of the task.

Parameters

description	The new task description.

Definition at line 78 of file Task.java.

```
00078                                                   {
00079         this.description = description;
00080     }
```

8.19.3.8 setStatus()

```
void com.projectmanagement.lib.models.Task.setStatus (
            TaskStatus status)
```

Sets the status of the task.

Parameters

status	The new task status.

Definition at line 94 of file Task.java.

```
00094                                               {
00095         this.status = status;
00096     }
```

Referenced by com.projectmanagement.lib.storage.MySQLTaskRepository.findAll(), com.projectmanagement.lib.storage.SQLiteTaskR
com.projectmanagement.lib.storage.MySQLTaskRepository.findById(), com.projectmanagement.lib.storage.SQLiteTaskRepository.find
and com.projectmanagement.lib.services.TaskService.updateTaskStatus().

8.19.3.9 setTitle()

```
void com.projectmanagement.lib.models.Task.setTitle (
            String title)
```

Sets the title of the task.

Parameters

title	The new task title.

Definition at line 62 of file Task.java.

```
00062                                       {
00063         this.title = title;
00064     }
```

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/models/Task.java

8.20 com.projectmanagement.lib.services.TaskService Class Reference

Public Member Functions

- TaskService (IRepository< Task > taskRepository)
- void registerTask (String id, String title, String description)
- void assignTaskToUser (String taskId, User assignedUser)
- void updateTaskStatus (String taskId, TaskStatus newStatus)
- List< Task > getAllTasks ()

8.20.1 Detailed Description

Interfaces business constraints and operations related precisely to Tasks.

Executes core routines spanning Task Assignment, detail configuration, and Status tracking. Accepts an `IRepository<Task>` parameter compliant with the core Dependency Injection rule.

Definition at line 13 of file TaskService.java.

8.20.2 Constructor & Destructor Documentation

8.20.2.1 TaskService()

```
com.projectmanagement.lib.services.TaskService.TaskService (
            IRepository< Task > taskRepository)
```

Formulates a TaskService requiring an explicit storage implementation.

Parameters

taskRepository	The storage backend (dependency string) connected for CRUD.

Definition at line 24 of file TaskService.java.

```
00024                                                   {
00025         this.taskRepository = taskRepository;
00026     }
```

8.20.3 Member Function Documentation

8.20.3.1 assignTaskToUser()

```
void com.projectmanagement.lib.services.TaskService.assignTaskToUser (
            String taskId,
            User assignedUser)
```

Allocates an established User entity onto a registered target Task.

Parameters

taskId	Key string pointing to the persistent Task.
assignedUser	Complete User object structure inheriting responsibility for this Task.

Exceptions

IllegalArgumentException	Triggers when either parameter is essentially unallocated or empty.

Definition at line 50 of file TaskService.java.

```
00050                                                                   {
00051         if (taskId == null || assignedUser == null) {
00052             throw new IllegalArgumentException("Target Task ID and the target User assignment cannot
      be vacant sequences.");
00053         }
00054
00055         Task trackingTask = taskRepository.findById(taskId);
00056
00057         if (trackingTask != null) {
00058             trackingTask.setAssignedUser(assignedUser);
00059             taskRepository.update(trackingTask);
00060         } else {
00061              throw new IllegalArgumentException("Target Task does not exist.");
00062         }
00063     }
```

References com.projectmanagement.lib.models.Task.setAssignedUser().

8.20.3.2 getAllTasks()

```
List< Task > com.projectmanagement.lib.services.TaskService.getAllTasks ()
```

Retrieves all registered tasks.

Returns

A list of all stored Task objects.

Definition at line 90 of file TaskService.java.

```
00090                                       {
00091         return taskRepository.findAll();
00092     }
```

8.20.3.3 registerTask()

```
void com.projectmanagement.lib.services.TaskService.registerTask (
            String id,
            String title,
            String description)
```

Instantiates a blank Task form and inserts it immediately into the operational repository.

Parameters

id	The unique Task string identifier.
title	Distinct name characterizing the Task objective.
description	Narrative outlining the core deliverables of this Task.

Exceptions

IllegalArgumentException	Raised when any critical task elements are effectively nonexistent.

Definition at line 35 of file TaskService.java.

```
00035                                                                                   {
00036         if (id == null || title == null || title.isEmpty() || description == null) {
00037             throw new IllegalArgumentException("Task attributes ID, Title, and Description are
      explicitly required.");
00038         }
00039
00040         Task newTask = new Task(id, title, description);
00041         taskRepository.save(newTask);
00042     }
```

8.20.3.4 updateTaskStatus()

```
void com.projectmanagement.lib.services.TaskService.updateTaskStatus (
            String taskId,
            TaskStatus newStatus)
```

Pushes a status transition onto an existing Task.

Parameters

taskId	Reference string locating the Task in progress.
newStatus	Enum value directing the new tracking state (e.g., TODO, IN_PROGRESS, DONE).

Exceptions

IllegalArgumentException	Returns if Task ID or Target Status params are completely unbound.

Definition at line 71 of file TaskService.java.

```
00071                                                                                   {
00072         if (taskId == null || newStatus == null) {
00073             throw new IllegalArgumentException("Task ID and new transition state are required
      constraints.");
00074         }
00075
00076         Task operatingTask = taskRepository.findById(taskId);
00077
00078         if (operatingTask != null) {
00079             operatingTask.setStatus(newStatus);
00080             taskRepository.update(operatingTask);
00081         } else {
00082             throw new IllegalArgumentException("Could not identify the selected Task.");
00083         }
00084     }
```

References com.projectmanagement.lib.models.Task.setStatus().

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/services/TaskService.java

8.21 com.projectmanagement.lib.models.TaskStatus Enum Reference

Public Attributes

- TODO
- IN_PROGRESS
- DONE

8.21.1 Detailed Description

Represents the current status of a task.

Definition at line 6 of file TaskStatus.java.

8.21.2 Member Data Documentation

8.21.2.1 DONE

`com.projectmanagement.lib.models.TaskStatus.DONE`

Indicates that the task has been completed.

Definition at line 20 of file TaskStatus.java.

Referenced by com.projectmanagement.lib.services.ReportService.generateTaskStatusReport().

8.21.2.2 IN_PROGRESS

`com.projectmanagement.lib.models.TaskStatus.IN_PROGRESS`

Indicates that the task is currently being worked on.

Definition at line 15 of file TaskStatus.java.

Referenced by com.projectmanagement.lib.services.ReportService.generateTaskStatusReport().

8.21.2.3 TODO

`com.projectmanagement.lib.models.TaskStatus.TODO`

Indicates that the task has not been started yet.

Definition at line 10 of file TaskStatus.java.

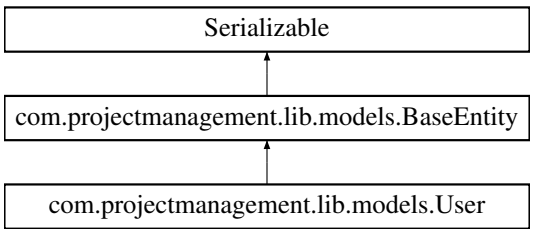
Referenced by com.projectmanagement.lib.services.ReportService.generateTaskStatusReport(), com.projectmanagement.lib.models and com.projectmanagement.lib.models.Task.Task().

The documentation for this enum was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/models/TaskStatus.java

8.22 com.projectmanagement.lib.models.User Class Reference

Inheritance diagram for com.projectmanagement.lib.models.User:



Public Member Functions

- User ()
- User (String id, String username, String email, String password)
- String getUsername ()
- void setUsername (String username)
- String getEmail ()
- void setEmail (String email)
- String getPassword ()
- void setPassword (String password)
- void displayDetails ()

Public Member Functions inherited from com.projectmanagement.lib.models.BaseEntity

- BaseEntity ()
- BaseEntity (String id)
- String getId ()
- void setId (String id)
- LocalDateTime getCreatedDate ()
- void setCreatedDate (LocalDateTime createdDate)

8.22.1 Detailed Description

Represents a user in the project management system.

Inherits from BaseEntity and encapsulates user-specific data like username and email.

Definition at line 7 of file User.java.

8.22.2 Constructor & Destructor Documentation

8.22.2.1 User() [1/2]

```
com.projectmanagement.lib.models.User.User ()
```

Default constructor for User.

Definition at line 26 of file User.java.

```
00026                    {
00027         super();
00028     }
```

8.22.2.2 User() [2/2]

```
com.projectmanagement.lib.models.User.User (
            String id,
            String username,
            String email,
            String password)
```

Parameterized constructor for User.

Parameters

id	The unique identifier of the user.
username	The username of the user.
email	The email address of the user.
password	The password for the user account.

Definition at line 37 of file User.java.

```
00037                                                                    {
00038         super(id);
00039         this.username = username;
00040         this.email = email;
00041         this.password = password;
00042     }
```

8.22.3 Member Function Documentation

8.22.3.1 displayDetails()

```
void com.projectmanagement.lib.models.User.displayDetails ()
```

Displays the details of the user.

Implementation of the abstract method from BaseEntity.

Reimplemented from com.projectmanagement.lib.models.BaseEntity.

Definition at line 97 of file User.java.

```
00097                                   {
00098         System.out.println("User ID: " + getId() + ", Username: " + username + ", Email: " + email);
00099     }
```

References com.projectmanagement.lib.models.BaseEntity.getId().

8.22.3.2 getEmail()

```
String com.projectmanagement.lib.models.User.getEmail ()
```

Gets the email address.

Returns

The email address.

Definition at line 64 of file User.java.

```
00064                                   {
00065         return email;
00066     }
```

Referenced by com.projectmanagement.lib.storage.MySQLUserRepository.save(), com.projectmanagement.lib.storage.SQLiteUserR
com.projectmanagement.lib.storage.MySQLUserRepository.update(), and com.projectmanagement.lib.storage.SQLiteUserRepository

8.22.3.3 getPassword()

```
String com.projectmanagement.lib.models.User.getPassword ()
```

Gets the user's password.

Returns

The password.

Definition at line 80 of file User.java.

```
00080                                   {
00081         return password;
00082     }
```

Referenced by com.projectmanagement.lib.storage.MySQLUserRepository.save(), com.projectmanagement.lib.storage.SQLiteUserR
com.projectmanagement.lib.storage.MySQLUserRepository.update(), and com.projectmanagement.lib.storage.SQLiteUserRepository

8.22.3.4 getUsername()

```
String com.projectmanagement.lib.models.User.getUsername ()
```

Gets the username.

Returns

The username.

Definition at line 48 of file User.java.

```
00048                                   {
00049         return username;
00050     }
```

Referenced by com.projectmanagement.lib.storage.MySQLUserRepository.save(), com.projectmanagement.lib.storage.SQLiteUserR
com.projectmanagement.lib.storage.MySQLUserRepository.update(), and com.projectmanagement.lib.storage.SQLiteUserRepository

8.22.3.5 setEmail()

```
void com.projectmanagement.lib.models.User.setEmail (
            String email)
```

Sets the email address.

Parameters

email	The new email address.

Definition at line 72 of file User.java.

```
00072                                           {
00073         this.email = email;
00074     }
```

8.22.3.6 setPassword()

```
void com.projectmanagement.lib.models.User.setPassword (
            String password)
```

Sets the user's password.

Parameters

password	The new password.

Definition at line 88 of file User.java.

```
00088                                           {
00089         this.password = password;
00090     }
```

8.22.3.7 setUsername()

```
void com.projectmanagement.lib.models.User.setUsername (
            String username)
```

Sets the username.

Parameters

username	The new username.

Definition at line 56 of file User.java.

```
00056                                           {
00057         this.username = username;
00058     }
```

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/models/User.java

8.23 com.projectmanagement.lib.services.UserService Class Reference

Public Member Functions

- UserService (IRepository< User > userRepository)
- void registerUser (String id, String username, String email, String plainPassword)
- User login (String username, String plainPassword)
- List< User > getAllUsers ()

8.23.1 Detailed Description

Provides business logic and validation for User operations.

Implements functionalities such as User Registration, Login and Password Hashing using SHA-256. Follows the Dependency Injection principle by accepting an IRepository instance.

Definition at line 14 of file UserService.java.

8.23.2 Constructor & Destructor Documentation

8.23.2.1 UserService()

```
com.projectmanagement.lib.services.UserService.UserService (
            IRepository< User > userRepository)
```

Constructs a new UserService with the designated User repository.

Parameters

userRepository	The storage implementation (dependency) to be injected.

Definition at line 26 of file UserService.java.

```
00026                                                   {
00027         this.userRepository = userRepository;
00028     }
```

8.23.3 Member Function Documentation

8.23.3.1 getAllUsers()

```
List< User > com.projectmanagement.lib.services.UserService.getAllUsers ()
```

Retrieves all registered users from the repository.

Returns

List of all User instances in storage.

Definition at line 80 of file UserService.java.

```
00080                                   {
00081         return userRepository.findAll();
00082     }
```

8.23.3.2 login()

```
User com.projectmanagement.lib.services.UserService.login (
            String username,
            String plainPassword)
```

Authenticates a user attempting to log in.

Retrieves all active users and verifies if the username and securely hashed password match a system record.

Parameters

username	The username attempting login.
plainPassword	The unencrypted password provided during login.

Returns

The authenticated User object if credentials are correct, null otherwise.

Definition at line 56 of file UserService.java.

```
00056                                                                   {
00057         if (username == null || plainPassword == null || plainPassword.isEmpty()) {
00058             return null;
00059         }
00060
00061         String targetHash = hashPassword(plainPassword);
00062         List<User> allUsers = userRepository.findAll();
00063
00064         // This is a basic implementation; realistic systems might query the repository directly via
      username
00065         if (allUsers != null) {
00066             for (User user : allUsers) {
00067                 if (user.getUsername().equals(username) && user.getPassword().equals(targetHash)) {
00068                     return user; // Successful login
00069                 }
00070             }
00071         }
00072
00073         return null; // Login failed
00074     }
```

8.23.3.3 registerUser()

```
void com.projectmanagement.lib.services.UserService.registerUser (
            String id,
            String username,
            String email,
            String plainPassword)
```

Registers a new user in the system.

Securely hashes the provided password before storing the User object.

Parameters

id	The unique identifier for the new user.
username	The desired username.
email	The user's email address.
plainPassword	The unencrypted password to be hashed and stored.

Exceptions

IllegalArgumentException	If any required parameter is null or empty.

Definition at line 39 of file UserService.java.

```
00039                                                                                                   {
00040         if (id == null || username == null || email == null || plainPassword == null ||
      plainPassword.isEmpty()) {
00041             throw new IllegalArgumentException("Registration fields cannot be null or empty.");
00042         }
00043
00044         String hashedPassword = hashPassword(plainPassword);
00045         User newUser = new User(id, username, email, hashedPassword);
00046         userRepository.save(newUser);
00047     }
```

The documentation for this class was generated from the following file:

- projectmanager-app/src/main/java/com/projectmanagement/lib/services/UserService.java

Chapter 9

File Documentation

9.1 projectmanager-app/src/main/java/com/projectmanagement/app/↩ Main.java File Reference

```
import com.projectmanagement.lib.models.User;
import com.projectmanagement.lib.services.ProjectService;
import com.projectmanagement.lib.services.TaskService;
import com.projectmanagement.lib.services.UserService;
import com.projectmanagement.lib.storage.RepositoryFactory;
import com.projectmanagement.lib.storage.StorageConfig;
import com.projectmanagement.lib.storage.StorageType;
import com.projectmanagement.lib.services.ReportService;
import org.jline.terminal.Terminal;
import org.jline.terminal.TerminalBuilder;
import org.jline.terminal.Attributes;
import org.jline.utils.InfoCmp.Capability;
import org.jline.utils.NonBlockingReader;
import java.util.Scanner;
import java.util.UUID;
```

Classes

- class com.projectmanagement.app.Main

Packages

- package com.projectmanagement.app

9.2 Main.java

Go to the documentation of this file.

```
00001 package com.projectmanagement.app;
00002
00003 import com.projectmanagement.lib.models.User;
00004 import com.projectmanagement.lib.services.ProjectService;
00005 import com.projectmanagement.lib.services.TaskService;
00006 import com.projectmanagement.lib.services.UserService;
00007 import com.projectmanagement.lib.storage.RepositoryFactory;
00008 import com.projectmanagement.lib.storage.StorageConfig;
00009 import com.projectmanagement.lib.storage.StorageType;
00010 import com.projectmanagement.lib.services.ReportService;
00011 import org.jline.terminal.Terminal;
00012 import org.jline.terminal.TerminalBuilder;
00013 import org.jline.terminal.Attributes;
00014 import org.jline.utils.InfoCmp.Capability;
00015 import org.jline.utils.NonBlockingReader;
00016
00017 import java.util.Scanner;
00018 import java.util.UUID;
00023 public class Main {
00024
00026     private static StorageConfig storageConfig = new StorageConfig(StorageType.BINARY_FILE);
00027
00029     private static RepositoryFactory repoFactory = new RepositoryFactory(storageConfig);
00030
00032     private static UserService userService = new UserService(repoFactory.getUserRepository());
00033
00035     private static ProjectService projectService = new
      ProjectService(repoFactory.getProjectRepository());
00036
00038     private static TaskService taskService = new TaskService(repoFactory.getTaskRepository());
00039
00041     private static ReportService reportService = new ReportService(projectService, taskService,
      userService);
00042
00044     private static User loggedInUser = null;
00045
00047     private static boolean isGuest = false;
00048
00050     private static Scanner scanner;
00051
00053     private static Terminal terminal;
00054
00060     public static void main(String[] args) {
00061         // Initialize scanner freshly. Allows unit tests to intercept System.in cleanly.
00062         scanner = new Scanner(System.in);
00063
00064         try {
00065             terminal = TerminalBuilder.builder().jna(true).system(true).build();
00066         } catch (Exception e) {
00067             terminal = null;
00068         }
00069
00070         boolean appRunning = true;
00071         while (appRunning) {
00072             if (loggedInUser == null && !isGuest) {
00073                 appRunning = renderAuthMenu();
00074             } else {
00075                 appRunning = renderMainMenu();
00076             }
00077         }
00078         System.out.println("Exiting application. Goodbye!");
00079         scanner.close();
00080     }
00081
00082     private static void reloadServices() {
00083         repoFactory = new RepositoryFactory(storageConfig);
00084         userService = new UserService(repoFactory.getUserRepository());
00085         projectService = new ProjectService(repoFactory.getProjectRepository());
00086         taskService = new TaskService(repoFactory.getTaskRepository());
00087         reportService = new ReportService(projectService, taskService, userService);
00088     }
00089
00090     private static String askInput(String prompt) {
00091         System.out.print(prompt);
00092         return scanner.nextLine().trim();
00093     }
00094
00095     private static String askPassword(String prompt) {
00096         System.out.print(prompt);
00097         return scanner.nextLine().trim();
00098     }
00099
```

```java
    private static int interactiveMenu(String title, String[] options) {
        if (terminal == null || terminal.getType().equals(Terminal.TYPE_DUMB) ||
terminal.getType().equals(Terminal.TYPE_DUMB_COLOR)) {
            return fallbackInteractiveMenu(title, options);
        }

        try {
            Attributes prevAttrs = terminal.enterRawMode();
            NonBlockingReader reader = terminal.reader();

            int selectedIndex = 0;
            while (true) {
                terminal.puts(Capability.clear_screen);
                terminal.writer().println("====================================\r");
                terminal.writer().println("   " + title + "\r");
                terminal.writer().println("====================================\r");
                terminal.writer().println("Use Arrow Keys or [W=Up, S=Down]. Enter to select.\r");
                terminal.writer().println("\r");

                for (int i = 0; i < options.length; i++) {
                    if (i == selectedIndex) {
                        terminal.writer().println(" -> " + options[i] + "\r");
                    } else {
                        terminal.writer().println("    " + options[i] + "\r");
                    }
                }
                terminal.writer().flush();

                int c = reader.read();
                if (c == 27) { // ESC sequence
                    int next = reader.read(50L);
                    if (next == 91) { // [
                        int arrow = reader.read(50L);
                        if (arrow == 65) { // Up
                            selectedIndex = (selectedIndex - 1 + options.length) % options.length;
                        } else if (arrow == 66) { // Down
                            selectedIndex = (selectedIndex + 1) % options.length;
                        }
                    }
                } else if (c == 'w' || c == 'W') {
                    selectedIndex = (selectedIndex - 1 + options.length) % options.length;
                } else if (c == 's' || c == 'S') {
                    selectedIndex = (selectedIndex + 1) % options.length;
                } else if (c == 13 || c == 10) { // Enter
                    terminal.setAttributes(prevAttrs);
                    // clear screen before exiting so the next output is clean
                    terminal.puts(Capability.clear_screen);
                    terminal.writer().flush();
                    return selectedIndex;
                }
            }
        } catch (Exception e) {
            return fallbackInteractiveMenu(title, options);
        }
    }

    private static int fallbackInteractiveMenu(String title, String[] options) {
        while (true) {
            System.out.println("\n====================================");
            System.out.println("   " + title);
            System.out.println("====================================");

            for (int i = 0; i < options.length; i++) {
                System.out.println((i + 1) + ". " + options[i]);
            }

            System.out.print("\nPlease select an option (1-" + options.length + "): ");
            String input = scanner.nextLine().trim();

            try {
                int choice = Integer.parseInt(input);
                if (choice >= 1 && choice <= options.length) {
                    return choice - 1; // 0-based index
                } else {
                    System.out.println("Invalid input! Please enter a valid number.");
                }
            } catch (NumberFormatException e) {
                System.out.println("Invalid character! Please enter numbers only.");
            }
        }
    }

    private static boolean renderAuthMenu() {
        String[] options = {
            "Login",
            "Register",
            "Guest Mode",
```

```
            "Change Storage Backend [" + storageConfig.getActiveStorageType().name() + "]",
            "Exit Application"
        };
        int selection = interactiveMenu("USER AUTHENTICATION", options);
        switch (selection) {
            case 0:
                handleLogin();
                break;
            case 1:
                handleRegister();
                break;
            case 2:
                isGuest = true;
                loggedInUser = null;
                System.out.println("Guest Mode activated.");
                pause();
                break;
            case 3:
                handleStorageSwap();
                break;
            case 4:
                return false;
        }
        return true;
    }

    private static boolean renderMainMenu() {
        String status = "Not Logged In";
        if (loggedInUser != null) status = "Logged in as: " + loggedInUser.getUsername();
        else if (isGuest) status = "Guest Mode Active";

        String[] menuOptions = {
            "Project Setup",
            "Task Assignment",
            "Progress Tracking",
            "Reporting",
            "Logout",
            "Exit Application"
        };
        int selection = interactiveMenu("MAIN MENU | " + status, menuOptions);
        switch (selection) {
            case 0:
                handleProjectSetup();
                break;
            case 1:
                handleTaskAssignment();
                break;
            case 2:
                 handleProgressTracking();
                 break;
            case 3:
                 handleReporting();
                 break;
            case 4:
                loggedInUser = null;
                isGuest = false;
                System.out.println("Logged out successfully.");
                pause();
                break;
            case 5:
                return false;
        }
        return true;
    }

    private static void handleLogin() {
        System.out.println("--- User Login ---");
        String username = askInput("Username: ");
        String password = askPassword("Password: ");

        User user = userService.login(username, password);
        if (user != null) {
            loggedInUser = user;
            isGuest = false;
            System.out.println("Login Successful! Welcome, " + user.getUsername() + ".");
        } else {
            System.out.println("Invalid username or password!");
        }
        pause();
    }

    private static void handleRegister() {
        System.out.println("--- New User Registration ---");
        String uname = askInput("Username: ");
        String email = askInput("Email: ");
        String password = askPassword("Password: ");

```

```
        try {
            userService.registerUser(UUID.randomUUID().toString(), uname, email, password);
            System.out.println("Registration successful! You can now login.");
        } catch (Exception e) {
            System.out.println("Registration failed: " + e.getMessage());
        }
        pause();
    }

    private static void handleStorageSwap() {
        String[] storageChoices = {"BINARY_FILE", "SQLITE", "MYSQL", "Cancel"};
        int choice = interactiveMenu("STORAGE BACKEND SELECTION", storageChoices);

        if (choice < 3) {
            StorageType newType = StorageType.values()[choice];
            storageConfig.setActiveStorageType(newType);
            reloadServices();
            System.out.println("Storage backend changed successfully: " + newType.name());
            pause();
        }
    }

    private static void handleProjectSetup() {
        boolean back = false;
        while (!back) {
            String[] options = {
                "Create Project",
                "View All Projects",
                "Back to Main Menu"
            };
            int selection = interactiveMenu("PROJECT SETUP", options);
            switch (selection) {
                case 0:
                    System.out.println("\n--- Create New Project ---");
                    String pName = askInput("Project Name: ");
                    String pDesc = askInput("Project Description: ");
                    try {
                        projectService.createProject(UUID.randomUUID().toString(), pName, pDesc);
                        System.out.println("Project created successfully!");
                    } catch (Exception e) {
                        System.out.println("Error: " + e.getMessage());
                    }
                    pause();
                    break;
                case 1:
                    System.out.println("\n--- All Projects ---");
                    java.util.List<com.projectmanagement.lib.models.Project> projects =
projectService.getAllProjects();
                    if (projects == null || projects.isEmpty()) {
                        System.out.println("No projects found.");
                    } else {
                        for (com.projectmanagement.lib.models.Project p : projects) {
                            p.displayDetails();
                            System.out.println("-");
                        }
                    }
                    pause();
                    break;
                case 2:
                    back = true;
                    break;
            }
        }
    }

    private static void handleTaskAssignment() {
        boolean back = false;
        while (!back) {
            String[] options = {
                "Create New Task & Add to Project",
                "Assign User to Task",
                "View All Tasks",
                "Back to Main Menu"
            };
            int selection = interactiveMenu("TASK MANAGEMENT", options);
            switch (selection) {
                case 0:
                    System.out.println("\n--- Create Task ---");
                    java.util.List<com.projectmanagement.lib.models.Project> projs =
projectService.getAllProjects();
                    if (projs == null || projs.isEmpty()) {
                        System.out.println("Please create a Project first!");
                        pause();
                        break;
                    }
                    System.out.println("Available Projects:");
                    for (int i = 0; i < projs.size(); i++) {
```

```
                        System.out.println((i+1) + ". " + projs.get(i).getName());
                    }
                    String pIndexStr = askInput("Select Project Number: ");
                    try {
                        int idx = Integer.parseInt(pIndexStr) - 1;
                        if (idx >= 0 && idx < projs.size()) {
                            String tTitle = askInput("Task Title: ");
                            String tDesc = askInput("Task Description: ");
                            com.projectmanagement.lib.models.Task newTask = new com.projectmanagement.lib.models.Task(UUID.randomUUID().toString(), tTitle, tDesc);
                            taskService.registerTask(newTask.getId(), tTitle, tDesc);
                            projectService.addTaskToProject(projs.get(idx).getId(), newTask);
                            System.out.println("Task created and added to Project.");
                        } else {
                            System.out.println("Invalid selection.");
                        }
                    } catch (Exception e) {
                        System.out.println("Error: " + e.getMessage());
                    }
                    pause();
                    break;
                case 1:
                    System.out.println("\n--- Assign User to Task ---");
                    java.util.List<com.projectmanagement.lib.models.Task> allTasks = taskService.getAllTasks();
                    java.util.List<User> allUsers = userService.getAllUsers();
                    if (allTasks == null || allTasks.isEmpty() || allUsers == null || allUsers.isEmpty()) {
                        System.out.println("You need both Tasks and Users in the system to assign them.");
                        pause();
                        break;
                    }
                    System.out.println("Tasks:");
                    for (int i=0; i<allTasks.size(); i++) {
                        System.out.println((i+1) + ". " + allTasks.get(i).getTitle() + " (Status: " + allTasks.get(i).getStatus() + ")");
                    }
                    String tIdxStr = askInput("Select Task Number: ");

                    System.out.println("Users:");
                    for (int i=0; i<allUsers.size(); i++) {
                        System.out.println((i+1) + ". " + allUsers.get(i).getUsername());
                    }
                    String uIdxStr = askInput("Select User Number: ");

                    try {
                        int currentTaskIndex = Integer.parseInt(tIdxStr) - 1;
                        int currentUserIndex = Integer.parseInt(uIdxStr) - 1;
                        taskService.assignTaskToUser(allTasks.get(currentTaskIndex).getId(), allUsers.get(currentUserIndex));
                        System.out.println("Task successfully assigned.");
                    } catch (Exception e) {
                        System.out.println("Error assigning task. Check inputs.");
                    }
                    pause();
                    break;
                case 2:
                    System.out.println("\n--- All Tasks ---");
                    java.util.List<com.projectmanagement.lib.models.Task> tasks = taskService.getAllTasks();
                    if (tasks == null || tasks.isEmpty()) {
                        System.out.println("No tasks found.");
                    } else {
                        for (com.projectmanagement.lib.models.Task t : tasks) {
                            t.displayDetails();
                            System.out.println("-");
                        }
                    }
                    pause();
                    break;
                case 3:
                    back = true;
                    break;
            }
        }
    }

    private static void handleProgressTracking() {
        boolean back = false;
        while (!back) {
            String[] options = {
                "Update Task Status",
                "Back to Main Menu"
            };
            int selection = interactiveMenu("PROGRESS TRACKING", options);
            switch (selection) {
```

```
                case 0:
                    System.out.println("\n--- Update Task Status ---");
                    java.util.List<com.projectmanagement.lib.models.Task> tasks = taskService.getAllTasks();
                    if(tasks == null || tasks.isEmpty()) {
                        System.out.println("No tasks available.");
                        pause();
                        break;
                    }
                    for (int i = 0; i < tasks.size(); i++) {
                        System.out.println((i+1) + ". " + tasks.get(i).getTitle() + " [" + tasks.get(i).getStatus() + "]");
                    }
                    String tIdxStr = askInput("Select Task Number: ");

                    System.out.println("1. TODO\n2. IN_PROGRESS\n3. DONE");
                    String sIdxStr = askInput("Select New Status (1-3): ");

                    try {
                        int tIdx = Integer.parseInt(tIdxStr) - 1;
                        int sIdx = Integer.parseInt(sIdxStr);
                        com.projectmanagement.lib.models.TaskStatus newStatus = com.projectmanagement.lib.models.TaskStatus.TODO;
                        if(sIdx == 2) newStatus = com.projectmanagement.lib.models.TaskStatus.IN_PROGRESS;
                        if(sIdx == 3) newStatus = com.projectmanagement.lib.models.TaskStatus.DONE;

                        taskService.updateTaskStatus(tasks.get(tIdx).getId(), newStatus);
                        System.out.println("Task status updated to " + newStatus.name());
                    } catch (Exception e) {
                        System.out.println("Invalid input!");
                    }
                    pause();
                    break;
                case 1:
                    back = true;
                    break;
            }
        }
    }

    private static void handleReporting() {
        boolean back = false;
        while (!back) {
            String[] options = {
                "Generate General System Report",
                "Generate Task Status Report",
                "Back to Main Menu"
            };
            int selection = interactiveMenu("REPORTING", options);
            switch (selection) {
                case 0:
                    reportService.generateGeneralSystemReport();
                    pause();
                    break;
                case 1:
                    reportService.generateTaskStatusReport();
                    pause();
                    break;
                case 2:
                    back = true;
                    break;
            }
        }
    }

    private static void pause() {
        System.out.println("\nPress [ENTER] to continue...");
        scanner.nextLine();
    }
}
```

9.3 projectmanager-app/src/main/java/com/projectmanagement/lib/models/BaseEntity.java File Reference

```
import java.time.LocalDateTime;
import java.io.Serializable;
```

Classes

- class com.projectmanagement.lib.models.BaseEntity

Packages

- package com.projectmanagement.lib.models

9.4 BaseEntity.java

Go to the documentation of this file.

```
00001 package com.projectmanagement.lib.models;
00002
00003 import java.time.LocalDateTime;
00004 import java.io.Serializable;
00005
00010 public abstract class BaseEntity implements Serializable {
00014     private static final long serialVersionUID = 1L;
00018     private String id;
00019
00023     private LocalDateTime createdDate;
00024
00029     public BaseEntity() {
00030         this.createdDate = LocalDateTime.now();
00031     }
00032
00037     public BaseEntity(String id) {
00038         this();
00039         this.id = id;
00040     }
00041
00046     public String getId() {
00047         return id;
00048     }
00049
00054     public void setId(String id) {
00055         this.id = id;
00056     }
00057
00062     public LocalDateTime getCreatedDate() {
00063         return createdDate;
00064     }
00065
00070     public void setCreatedDate(LocalDateTime createdDate) {
00071         this.createdDate = createdDate;
00072     }
00073
00078     public abstract void displayDetails();
00079 }
```

9.5 projectmanager-app/src/main/java/com/projectmanagement/lib/↩ models/Project.java File Reference

```
import java.util.ArrayList;
import java.util.List;
```

Classes

- class com.projectmanagement.lib.models.Project

Packages

- package com.projectmanagement.lib.models

9.6 Project.java

Go to the documentation of this file.

```
package com.projectmanagement.lib.models;

import java.util.ArrayList;
import java.util.List;

public class Project extends BaseEntity {
    private String name;

    private String description;

    private List<Task> tasks;

    public Project() {
        super();
        this.tasks = new ArrayList<>();
    }

    public Project(String id, String name, String description) {
        super(id);
        this.name = name;
        this.description = description;
        this.tasks = new ArrayList<>();
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public List<Task> getTasks() {
        return tasks;
    }

    public void setTasks(List<Task> tasks) {
        this.tasks = tasks;
    }

    public void addTask(Task task) {
        this.tasks.add(task);
    }

    public void removeTask(Task task) {
        this.tasks.remove(task);
    }

    @Override
    public void displayDetails() {
        System.out.println("Project ID: " + getId() + ", Name: " + name);
        System.out.println("Description: " + description);
        System.out.println("Total Tasks: " + tasks.size());
    }
}
```

9.7 projectmanager-app/src/main/java/com/projectmanagement/lib/↩ models/Task.java File Reference

Classes

- class com.projectmanagement.lib.models.Task

Packages

- package com.projectmanagement.lib.models

9.8 Task.java

Go to the documentation of this file.

```
00001 package com.projectmanagement.lib.models;
00002
00007 public class Task extends BaseEntity {
00011     private String title;
00012
00016     private String description;
00017
00021     private TaskStatus status;
00022
00026     private User assignedUser;
00027
00032     public Task() {
00033         super();
00034         this.status = TaskStatus.TODO;
00035     }
00036
00043     public Task(String id, String title, String description) {
00044         super(id);
00045         this.title = title;
00046         this.description = description;
00047         this.status = TaskStatus.TODO;
00048     }
00049
00054     public String getTitle() {
00055         return title;
00056     }
00057
00062     public void setTitle(String title) {
00063         this.title = title;
00064     }
00065
00070     public String getDescription() {
00071         return description;
00072     }
00073
00078     public void setDescription(String description) {
00079         this.description = description;
00080     }
00081
00086     public TaskStatus getStatus() {
00087         return status;
00088     }
00089
00094     public void setStatus(TaskStatus status) {
00095         this.status = status;
00096     }
00097
00102     public User getAssignedUser() {
00103         return assignedUser;
00104     }
00105
00110     public void setAssignedUser(User assignedUser) {
00111         this.assignedUser = assignedUser;
00112     }
00113
00118     @Override
00119     public void displayDetails() {
00120         System.out.println("Task ID: " + getId() + ", Title: " + title);
00121         System.out.println("Status: " + status);
00122         if (assignedUser != null) {
00123             System.out.println("Assigned To: " + assignedUser.getUsername());
00124         } else {
00125             System.out.println("Assigned To: Unassigned");
00126         }
00127     }
00128 }
```

9.9 projectmanager-app/src/main/java/com/projectmanagement/lib/models/TaskStatus.java File Reference

Classes

- enum com.projectmanagement.lib.models.TaskStatus

Packages

- package com.projectmanagement.lib.models

9.10 TaskStatus.java

Go to the documentation of this file.

```
00001 package com.projectmanagement.lib.models;
00002
00006 public enum TaskStatus {
00010     TODO,
00011
00015     IN_PROGRESS,
00016
00020     DONE
00021 }
```

9.11 projectmanager-app/src/main/java/com/projectmanagement/lib/models/User.java File Reference

Classes

- class com.projectmanagement.lib.models.User

Packages

- package com.projectmanagement.lib.models

9.12 User.java

Go to the documentation of this file.

```
00001 package com.projectmanagement.lib.models;
00002
00007 public class User extends BaseEntity {
00011     private String username;
00012
00016     private String email;
00017
00021     private String password;
00022
00026     public User() {
00027         super();
00028     }
00029
00037     public User(String id, String username, String email, String password) {
00038         super(id);
00039         this.username = username;
```

```
        this.email = email;
        this.password = password;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    @Override
    public void displayDetails() {
        System.out.println("User ID: " + getId() + ", Username: " + username + ", Email: " + email);
    }
}
```

9.13 projectmanager-app/src/main/java/com/projectmanagement/lib/↩ services/ProjectService.java File Reference

```
import com.projectmanagement.lib.models.Project;
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.storage.IRepository;
import java.util.List;
```

Classes

- class com.projectmanagement.lib.services.ProjectService

Packages

- package com.projectmanagement.lib.services

9.14 ProjectService.java

Go to the documentation of this file.

```
package com.projectmanagement.lib.services;

import com.projectmanagement.lib.models.Project;
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.storage.IRepository;
import java.util.List;

public class ProjectService {

    private final IRepository<Project> projectRepository;
```

```
    public ProjectService(IRepository<Project> projectRepository) {
        this.projectRepository = projectRepository;
    }

    public void createProject(String id, String name, String description) {
        if (id == null || name == null || name.isEmpty() || description == null) {
            throw new IllegalArgumentException("Project initialization values cannot be void or blank.");
        }

        Project newProject = new Project(id, name, description);
        projectRepository.save(newProject);
    }

    public void addTaskToProject(String projectId, Task task) {
        if (projectId == null || task == null) {
            throw new IllegalArgumentException("Target Project identifier and Task components must be provided.");
        }

        Project existingProject = projectRepository.findById(projectId);
        if (existingProject == null) {
            throw new IllegalArgumentException("Targeted Project does not exist.");
        }

        existingProject.addTask(task);
        projectRepository.update(existingProject); // Merges the modified relation structure back to the Repo
    }

    public List<Project> getAllProjects() {
        return projectRepository.findAll();
    }
}
```

9.15 projectmanager-app/src/main/java/com/projectmanagement/lib/services/ReportService.java File Reference

```
import com.projectmanagement.lib.models.Project;
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.User;
import com.projectmanagement.lib.models.TaskStatus;
import java.util.List;
```

Classes

- class com.projectmanagement.lib.services.ReportService

Packages

- package com.projectmanagement.lib.services

9.16 ReportService.java

Go to the documentation of this file.

```
package com.projectmanagement.lib.services;

import com.projectmanagement.lib.models.Project;
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.User;
import com.projectmanagement.lib.models.TaskStatus;
```

```java

import java.util.List;

public class ReportService {

    private final ProjectService projectService;
    private final TaskService taskService;
    private final UserService userService;

    public ReportService(ProjectService projectService, TaskService taskService, UserService userService) {
        this.projectService = projectService;
        this.taskService = taskService;
        this.userService = userService;
    }

    public void generateGeneralSystemReport() {
        System.out.println("\n=====================================");
        System.out.println("       GENERAL SYSTEM REPORT         ");
        System.out.println("=====================================");

        List<Project> projects = projectService.getAllProjects();
        List<Task> tasks = taskService.getAllTasks();
        List<User> users = userService.getAllUsers();

        System.out.println("Total Registered Users  : " + (users != null ? users.size() : 0));
        System.out.println("Total Managed Projects  : " + (projects != null ? projects.size() : 0));
        System.out.println("Total Tracked Tasks     : " + (tasks != null ? tasks.size() : 0));

        if (projects != null && !projects.isEmpty()) {
            System.out.println("\n--- Projects Breakdown ---");
            for (Project project : projects) {
                System.out.println(" [Project] " + project.getName() + " (Contains " + project.getTasks().size() + " tasks)");
            }
        }
        System.out.println("=====================================\n");
    }

    public void generateTaskStatusReport() {
        System.out.println("\n=====================================");
        System.out.println("         TASK STATUS REPORT          ");
        System.out.println("=====================================");

        List<Task> tasks = taskService.getAllTasks();
        if (tasks == null || tasks.isEmpty()) {
            System.out.println("No tasks found in the system.");
            System.out.println("=====================================\n");
            return;
        }

        long todo = tasks.stream().filter(t -> t.getStatus() == TaskStatus.TODO).count();
        long inProgress = tasks.stream().filter(t -> t.getStatus() == TaskStatus.IN_PROGRESS).count();
        long done = tasks.stream().filter(t -> t.getStatus() == TaskStatus.DONE).count();

        System.out.println("Total Tasks     : " + tasks.size());
        System.out.println("TODO            : " + todo);
        System.out.println("IN_PROGRESS     : " + inProgress);
        System.out.println("DONE            : " + done);
        System.out.println("=====================================\n");
    }
}
```

9.17 projectmanager-app/src/main/java/com/projectmanagement/lib/services/TaskService.java File Reference

```java
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.TaskStatus;
import com.projectmanagement.lib.models.User;
import com.projectmanagement.lib.storage.IRepository;
import java.util.List;
```

Classes

- class com.projectmanagement.lib.services.TaskService

Packages

- package com.projectmanagement.lib.services

9.18 TaskService.java

Go to the documentation of this file.

```
package com.projectmanagement.lib.services;

import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.TaskStatus;
import com.projectmanagement.lib.models.User;
import com.projectmanagement.lib.storage.IRepository;
import java.util.List;

public class TaskService {

    private final IRepository<Task> taskRepository;

    public TaskService(IRepository<Task> taskRepository) {
        this.taskRepository = taskRepository;
    }

    public void registerTask(String id, String title, String description) {
        if (id == null || title == null || title.isEmpty() || description == null) {
            throw new IllegalArgumentException("Task attributes ID, Title, and Description are explicitly required.");
        }

        Task newTask = new Task(id, title, description);
        taskRepository.save(newTask);
    }

    public void assignTaskToUser(String taskId, User assignedUser) {
        if (taskId == null || assignedUser == null) {
            throw new IllegalArgumentException("Target Task ID and the target User assignment cannot be vacant sequences.");
        }

        Task trackingTask = taskRepository.findById(taskId);

        if (trackingTask != null) {
            trackingTask.setAssignedUser(assignedUser);
            taskRepository.update(trackingTask);
        } else {
             throw new IllegalArgumentException("Target Task does not exist.");
        }
    }

    public void updateTaskStatus(String taskId, TaskStatus newStatus) {
        if (taskId == null || newStatus == null) {
             throw new IllegalArgumentException("Task ID and new transition state are required constraints.");
        }

        Task operatingTask = taskRepository.findById(taskId);

        if (operatingTask != null) {
            operatingTask.setStatus(newStatus);
            taskRepository.update(operatingTask);
        } else {
             throw new IllegalArgumentException("Could not identify the selected Task.");
        }
    }

    public List<Task> getAllTasks() {
        return taskRepository.findAll();
    }
}
```

9.19 projectmanager-app/src/main/java/com/projectmanagement/lib/↩ services/UserService.java File Reference

```
import com.projectmanagement.lib.models.User;
import com.projectmanagement.lib.storage.IRepository;
```

```
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.List;
```

Classes

- class com.projectmanagement.lib.services.UserService

Packages

- package com.projectmanagement.lib.services

9.20 UserService.java

Go to the documentation of this file.

```
00001 package com.projectmanagement.lib.services;
00002
00003 import com.projectmanagement.lib.models.User;
00004 import com.projectmanagement.lib.storage.IRepository;
00005 import java.nio.charset.StandardCharsets;
00006 import java.security.MessageDigest;
00007 import java.security.NoSuchAlgorithmException;
00008 import java.util.List;
00009
00014 public class UserService {
00015
00020     private final IRepository<User> userRepository;
00021
00026     public UserService(IRepository<User> userRepository) {
00027         this.userRepository = userRepository;
00028     }
00029
00039     public void registerUser(String id, String username, String email, String plainPassword) {
00040         if (id == null || username == null || email == null || plainPassword == null ||
      plainPassword.isEmpty()) {
00041             throw new IllegalArgumentException("Registration fields cannot be null or empty.");
00042         }
00043
00044         String hashedPassword = hashPassword(plainPassword);
00045         User newUser = new User(id, username, email, hashedPassword);
00046         userRepository.save(newUser);
00047     }
00048
00056     public User login(String username, String plainPassword) {
00057         if (username == null || plainPassword == null || plainPassword.isEmpty()) {
00058             return null;
00059         }
00060
00061         String targetHash = hashPassword(plainPassword);
00062         List<User> allUsers = userRepository.findAll();
00063
00064         // This is a basic implementation; realistic systems might query the repository directly via
      username
00065         if (allUsers != null) {
00066             for (User user : allUsers) {
00067                 if (user.getUsername().equals(username) && user.getPassword().equals(targetHash)) {
00068                     return user; // Successful login
00069                 }
00070             }
00071         }
00072
00073         return null; // Login failed
00074     }
00075
00080     public List<User> getAllUsers() {
00081         return userRepository.findAll();
00082     }
00083
00091     private String hashPassword(String password) {
```

```
        try {
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            byte[] encodedhash = digest.digest(password.getBytes(StandardCharsets.UTF_8));
            return bytesToHex(encodedhash);
        } catch (NoSuchAlgorithmException e) {
            throw new RuntimeException("SHA-256 algorithm not found.", e);
        }
    }

    private String bytesToHex(byte[] hash) {
        StringBuilder hexString = new StringBuilder(2 * hash.length);
        for (int i = 0; i < hash.length; i++) {
            String hex = Integer.toHexString(0xff & hash[i]);
            if (hex.length() == 1) {
                hexString.append('0');
            }
            hexString.append(hex);
        }
        return hexString.toString();
    }
}
```

9.21 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/BinaryProjectRepository.java File Reference

```
import com.projectmanagement.lib.models.Project;
import java.io.*;
import java.util.ArrayList;
import java.util.List;
```

Classes

- class com.projectmanagement.lib.storage.BinaryProjectRepository

Packages

- package com.projectmanagement.lib.storage

9.22 BinaryProjectRepository.java

Go to the documentation of this file.

```
package com.projectmanagement.lib.storage;

import com.projectmanagement.lib.models.Project;
import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class BinaryProjectRepository implements IRepository<Project> {

    private final String FILE_PATH = "projects.dat";

    @SuppressWarnings("unchecked")
    private List<Project> loadFromFile() {
        File file = new File(FILE_PATH);
        if (!file.exists()) {
            return new ArrayList<>();
        }
        try (FileInputStream fis = new FileInputStream(file);
             ObjectInputStream ois = new ObjectInputStream(fis)) {
            return (List<Project>) ois.readObject();
        } catch (Exception e) {
            return new ArrayList<>();
```

```
        }
    }

    private void saveToFile(List<Project> projects) {
        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(FILE_PATH))) {
            oos.writeObject(projects);
        } catch (IOException e) {
            System.err.println("Error writing binary file: " + e.getMessage());
        }
    }

    @Override
    public void save(Project entity) {
        List<Project> projects = loadFromFile();
        projects.add(entity);
        saveToFile(projects);
    }

    @Override
    public Project findById(String id) {
        List<Project> projects = loadFromFile();
        for (Project p : projects) {
            if (p.getId().equals(id)) {
                return p;
            }
        }
        return null;
    }

    @Override
    public List<Project> findAll() {
        return loadFromFile();
    }

    @Override
    public void update(Project entity) {
        List<Project> projects = loadFromFile();
        for (int i = 0; i < projects.size(); i++) {
            if (projects.get(i).getId().equals(entity.getId())) {
                projects.set(i, entity);
                break;
            }
        }
        saveToFile(projects);
    }

    @Override
    public void delete(String id) {
        List<Project> projects = loadFromFile();
        projects.removeIf(p -> p.getId().equals(id));
        saveToFile(projects);
    }
}
```

9.23 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/BinaryTaskRepository.java File Reference

```
import com.projectmanagement.lib.models.Task;
import java.io.*;
import java.util.ArrayList;
import java.util.List;
```

Classes

- class com.projectmanagement.lib.storage.BinaryTaskRepository

Packages

- package com.projectmanagement.lib.storage

9.24 BinaryTaskRepository.java

Go to the documentation of this file.

```
package com.projectmanagement.lib.storage;

import com.projectmanagement.lib.models.Task;
import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class BinaryTaskRepository implements IRepository<Task> {

    private final String FILE_PATH = "tasks.dat";

    @SuppressWarnings("unchecked")
    private List<Task> loadFromFile() {
        File file = new File(FILE_PATH);
        if (!file.exists()) {
            return new ArrayList<>();
        }
        try (FileInputStream fis = new FileInputStream(file);
             ObjectInputStream ois = new ObjectInputStream(fis)) {
            return (List<Task>) ois.readObject();
        } catch (Exception e) {
            return new ArrayList<>();
        }
    }

    private void saveToFile(List<Task> tasks) {
        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(FILE_PATH))) {
            oos.writeObject(tasks);
        } catch (IOException e) {
            System.err.println("Error writing binary file: " + e.getMessage());
        }
    }

    @Override
    public void save(Task entity) {
        List<Task> tasks = loadFromFile();
        tasks.add(entity);
        saveToFile(tasks);
    }

    @Override
    public Task findById(String id) {
        List<Task> tasks = loadFromFile();
        for (Task t : tasks) {
            if (t.getId().equals(id)) {
                return t;
            }
        }
        return null;
    }

    @Override
    public List<Task> findAll() {
        return loadFromFile();
    }

    @Override
    public void update(Task entity) {
        List<Task> tasks = loadFromFile();
        for (int i = 0; i < tasks.size(); i++) {
            if (tasks.get(i).getId().equals(entity.getId())) {
                tasks.set(i, entity);
                break;
            }
        }
        saveToFile(tasks);
    }

    @Override
    public void delete(String id) {
        List<Task> tasks = loadFromFile();
        tasks.removeIf(t -> t.getId().equals(id));
        saveToFile(tasks);
    }
}
```

9.25 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/BinaryUserRepository.java File Reference

```
import com.projectmanagement.lib.models.User;
import java.io.*;
import java.util.ArrayList;
import java.util.List;
```

Classes

- class com.projectmanagement.lib.storage.BinaryUserRepository

Packages

- package com.projectmanagement.lib.storage

9.26 BinaryUserRepository.java

Go to the documentation of this file.

```
00001 package com.projectmanagement.lib.storage;
00002
00003 import com.projectmanagement.lib.models.User;
00004 import java.io.*;
00005 import java.util.ArrayList;
00006 import java.util.List;
00007
00012 public class BinaryUserRepository implements IRepository<User> {
00013
00017     private final String FILE_PATH = "users.dat";
00018
00023     @SuppressWarnings("unchecked")
00024     private List<User> loadFromFile() {
00025         File file = new File(FILE_PATH);
00026         if (!file.exists()) {
00027             return new ArrayList<>();
00028         }
00029         try (FileInputStream fis = new FileInputStream(file);
00030              ObjectInputStream ois = new ObjectInputStream(fis)) {
00031             return (List<User>) ois.readObject();
00032         } catch (Exception e) {
00033             return new ArrayList<>();
00034         }
00035     }
00036
00041     private void saveToFile(List<User> users) {
00042         try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(FILE_PATH))) {
00043             oos.writeObject(users);
00044         } catch (IOException e) {
00045             System.err.println("Error writing binary file: " + e.getMessage());
00046         }
00047     }
00048
00053     @Override
00054     public void save(User entity) {
00055         List<User> users = loadFromFile();
00056         users.add(entity);
00057         saveToFile(users);
00058     }
00059
00065     @Override
00066     public User findById(String id) {
00067         List<User> users = loadFromFile();
00068         for (User u : users) {
00069             if (u.getId().equals(id)) {
00070                 return u;
00071             }
00072         }
```

```
        return null;
    }

    @Override
    public List<User> findAll() {
        return loadFromFile();
    }

    @Override
    public void update(User entity) {
        List<User> users = loadFromFile();
        for (int i = 0; i < users.size(); i++) {
            if (users.get(i).getId().equals(entity.getId())) {
                users.set(i, entity);
                break;
            }
        }
        saveToFile(users);
    }

    @Override
    public void delete(String id) {
        List<User> users = loadFromFile();
        users.removeIf(u -> u.getId().equals(id));
        saveToFile(users);
    }
}
```

9.27 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/IRepository.java File Reference

```
import java.util.List;
```

Classes

- interface com.projectmanagement.lib.storage.IRepository< T >

Packages

- package com.projectmanagement.lib.storage

9.28 IRepository.java

Go to the documentation of this file.

```
package com.projectmanagement.lib.storage;

import java.util.List;

public interface IRepository<T> {

    void save(T entity);

    T findById(String id);

    List<T> findAll();

    void update(T entity);

    void delete(String id);
}
```

9.29 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/MySQLProjectRepository.java File Reference

```
import com.projectmanagement.lib.models.Project;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;
```

Classes

- class com.projectmanagement.lib.storage.MySQLProjectRepository

Packages

- package com.projectmanagement.lib.storage

9.30 MySQLProjectRepository.java

Go to the documentation of this file.

```
00001 package com.projectmanagement.lib.storage;
00002
00003 import com.projectmanagement.lib.models.Project;
00004 import java.sql.*;
00005 import java.util.ArrayList;
00006 import java.util.List;
00007
00012 public class MySQLProjectRepository implements IRepository<Project> {
00013
00014     private static final String URL =
      "jdbc:mysql://localhost:3306/projectmanager?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true&serv
00015     private static final String USER_DB = "root";
00016     private static final String PASS_DB = "root";
00017
00021     public MySQLProjectRepository() {
00022         createTableIfNotExists();
00023     }
00024
00025     private void createTableIfNotExists() {
00026         String sql = "CREATE TABLE IF NOT EXISTS projects ("
00027                    + "id VARCHAR(255) PRIMARY KEY,"
00028                    + "name VARCHAR(255) NOT NULL,"
00029                    + "description TEXT"
00030                    + ");";
00031         String mappingSql = "CREATE TABLE IF NOT EXISTS project_tasks ("
00032                    + "project_id VARCHAR(255),"
00033                    + "task_id VARCHAR(255),"
00034                    + "PRIMARY KEY (project_id, task_id)"
00035                    + ");";
00036         try (Connection conn = getConnection();
00037              Statement stmt = conn.createStatement()) {
00038             stmt.execute(sql);
00039             stmt.execute(mappingSql);
00040         } catch (SQLException e) {
00041             System.err.println("Error creating projects table (MySQL): " + e.getMessage());
00042         }
00043     }
00044
00049     @Override
00050     public void save(Project entity) {
00051         String sql = "INSERT INTO projects(id, name, description) VALUES(?, ?, ?)";
00052         try (Connection conn = getConnection();
00053              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00054             pstmt.setString(1, entity.getId());
00055             pstmt.setString(2, entity.getName());
00056             pstmt.setString(3, entity.getDescription());
00057             pstmt.executeUpdate();
00058
```

```
            saveProjectTasks(conn, entity);
        } catch (SQLException e) {
            System.err.println("Error creating project: " + e.getMessage());
        }
    }

    private void saveProjectTasks(Connection conn, Project entity) throws SQLException {
        String delSql = "DELETE FROM project_tasks WHERE project_id = ?";
        try (PreparedStatement delStmt = conn.prepareStatement(delSql)) {
            delStmt.setString(1, entity.getId());
            delStmt.executeUpdate();
        }

        if (entity.getTasks() != null && !entity.getTasks().isEmpty()) {
            String insSql = "INSERT INTO project_tasks (project_id, task_id) VALUES (?, ?)";
            try (PreparedStatement insStmt = conn.prepareStatement(insSql)) {
                for (com.projectmanagement.lib.models.Task task : entity.getTasks()) {
                    insStmt.setString(1, entity.getId());
                    insStmt.setString(2, task.getId());
                    insStmt.executeUpdate();
                }
            }
        }
    }

    @Override
    public Project findById(String id) {
        String sql = "SELECT * FROM projects WHERE id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, id);
            ResultSet rs = pstmt.executeQuery();
            if (rs.next()) {
                Project project = new Project(
                    rs.getString("id"),
                    rs.getString("name"),
                    rs.getString("description")
                );
                loadProjectTasks(conn, project);
                return project;
            }
        } catch (SQLException e) {
            System.err.println("Error finding project by id: " + e.getMessage());
        }
        return null;
    }

    private void loadProjectTasks(Connection conn, Project project) throws SQLException {
        String taskSql = "SELECT task_id FROM project_tasks WHERE project_id = ?";
        try (PreparedStatement tpstmt = conn.prepareStatement(taskSql)) {
            tpstmt.setString(1, project.getId());
            ResultSet trs = tpstmt.executeQuery();
            MySQLTaskRepository taskRepo = new MySQLTaskRepository();
            while (trs.next()) {
                com.projectmanagement.lib.models.Task task =
taskRepo.findById(trs.getString("task_id"));
                if (task != null) {
                    project.addTask(task);
                }
            }
        }
    }

    @Override
    public List<Project> findAll() {
        List<Project> projects = new ArrayList<>();
        String sql = "SELECT * FROM projects";
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                Project project = new Project(
                    rs.getString("id"),
                    rs.getString("name"),
                    rs.getString("description")
                );
                loadProjectTasks(conn, project);
                projects.add(project);
            }
        } catch (SQLException e) {
            System.err.println("Error finding all projects: " + e.getMessage());
        }
        return projects;
    }

    @Override
    public void update(Project entity) {
```

```
        String sql = "UPDATE projects SET name = ?, description = ? WHERE id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, entity.getName());
            pstmt.setString(2, entity.getDescription());
            pstmt.setString(3, entity.getId());
            pstmt.executeUpdate();

            saveProjectTasks(conn, entity);
        } catch (SQLException e) {
            System.err.println("Error updating project: " + e.getMessage());
        }
    }

    @Override
    public void delete(String id) {
        String sql = "DELETE FROM projects WHERE id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, id);
            pstmt.executeUpdate();

            String delSql = "DELETE FROM project_tasks WHERE project_id = ?";
            try (PreparedStatement delStmt = conn.prepareStatement(delSql)) {
                delStmt.setString(1, id);
                delStmt.executeUpdate();
            }
        } catch (SQLException e) {
            System.err.println("Error deleting project: " + e.getMessage());
        }
    }

    protected Connection getConnection() throws SQLException { return DriverManager.getConnection(URL, USER_DB, PASS_DB); }
}
```

9.31 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/MySQLTaskRepository.java File Reference

```
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.TaskStatus;
import com.projectmanagement.lib.models.User;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;
```

Classes

- class com.projectmanagement.lib.storage.MySQLTaskRepository

Packages

- package com.projectmanagement.lib.storage

9.32 MySQLTaskRepository.java

Go to the documentation of this file.

```
package com.projectmanagement.lib.storage;

import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.TaskStatus;
import com.projectmanagement.lib.models.User;
```

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class MySQLTaskRepository implements IRepository<Task> {

    private static final String URL = "jdbc:mysql://localhost:3306/projectmanager?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true&serv
    private static final String USER_DB = "root";
    private static final String PASS_DB = "root";

    public MySQLTaskRepository() {
        createTableIfNotExists();
    }

    private void createTableIfNotExists() {
        String sql = "CREATE TABLE IF NOT EXISTS tasks ("
                   + "id VARCHAR(255) PRIMARY KEY,"
                   + "title VARCHAR(255) NOT NULL,"
                   + "description TEXT,"
                   + "status VARCHAR(50) NOT NULL,"
                   + "assigned_user_id VARCHAR(255)"
                   + ");";
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement()) {
            stmt.execute(sql);
        } catch (SQLException e) {
            System.err.println("Error creating tasks table (MySQL): " + e.getMessage());
        }
    }

    @Override
    public void save(Task entity) {
        String sql = "INSERT INTO tasks(id, title, description, status, assigned_user_id) VALUES(?, ?, ?, ?, ?)";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, entity.getId());
            pstmt.setString(2, entity.getTitle());
            pstmt.setString(3, entity.getDescription());
            pstmt.setString(4, entity.getStatus().name());
            pstmt.setString(5, entity.getAssignedUser() != null ? entity.getAssignedUser().getId() : null);
            pstmt.executeUpdate();
        } catch (SQLException e) {
            System.err.println("Error creating task: " + e.getMessage());
        }
    }

    @Override
    public Task findById(String id) {
        String sql = "SELECT * FROM tasks WHERE id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, id);
            ResultSet rs = pstmt.executeQuery();
            if (rs.next()) {
                Task task = new Task(
                    rs.getString("id"),
                    rs.getString("title"),
                    rs.getString("description")
                );
                task.setStatus(TaskStatus.valueOf(rs.getString("status")));
                String userId = rs.getString("assigned_user_id");
                if (userId != null) {
                    User assignee = new MySQLUserRepository().findById(userId);
                    task.setAssignedUser(assignee);
                }
                return task;
            }
        } catch (SQLException e) {
            System.err.println("Error finding task by id: " + e.getMessage());
        }
        return null;
    }

    @Override
    public List<Task> findAll() {
        List<Task> tasks = new ArrayList<>();
        String sql = "SELECT * FROM tasks";
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            MySQLUserRepository userRepo = new MySQLUserRepository();
            while (rs.next()) {
                Task task = new Task(
                    rs.getString("id"),
```

```
                    rs.getString("title"),
                    rs.getString("description")
                );
                task.setStatus(TaskStatus.valueOf(rs.getString("status")));
                String userId = rs.getString("assigned_user_id");
                if (userId != null) {
                    task.setAssignedUser(userRepo.findById(userId));
                }
                tasks.add(task);
            }
        } catch (SQLException e) {
            System.err.println("Error finding all tasks: " + e.getMessage());
        }
        return tasks;
    }

    @Override
    public void update(Task entity) {
        String sql = "UPDATE tasks SET title = ?, description = ?, status = ?, assigned_user_id = ? WHERE id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, entity.getTitle());
            pstmt.setString(2, entity.getDescription());
            pstmt.setString(3, entity.getStatus().name());
            pstmt.setString(4, entity.getAssignedUser() != null ? entity.getAssignedUser().getId() : null);
            pstmt.setString(5, entity.getId());
            pstmt.executeUpdate();
        } catch (SQLException e) {
            System.err.println("Error updating task: " + e.getMessage());
        }
    }

    @Override
    public void delete(String id) {
        String sql = "DELETE FROM tasks WHERE id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, id);
            pstmt.executeUpdate();
        } catch (SQLException e) {
            System.err.println("Error deleting task: " + e.getMessage());
        }
    }

    protected Connection getConnection() throws SQLException { return DriverManager.getConnection(URL, USER_DB, PASS_DB); }
}
```

9.33 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/MySQLUserRepository.java File Reference

```
import com.projectmanagement.lib.models.User;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;
```

Classes

- class com.projectmanagement.lib.storage.MySQLUserRepository

Packages

- package com.projectmanagement.lib.storage

9.34 MySQLUserRepository.java

Go to the documentation of this file.

```
package com.projectmanagement.lib.storage;

import com.projectmanagement.lib.models.User;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class MySQLUserRepository implements IRepository<User> {

    private static final String URL =
"jdbc:mysql://localhost:3306/projectmanager?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true&serv
    private static final String USER_DB = "root";
    private static final String PASS_DB = "root";

    public MySQLUserRepository() {
        createTableIfNotExists();
    }

    private void createTableIfNotExists() {
        String sql = "CREATE TABLE IF NOT EXISTS users ("
                   + "id VARCHAR(255) PRIMARY KEY,"
                   + "username VARCHAR(255) NOT NULL,"
                   + "email VARCHAR(255) NOT NULL,"
                   + "password VARCHAR(255) NOT NULL"
                   + ");";
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement()) {
            stmt.execute(sql);
        } catch (SQLException e) {
            System.err.println("Error creating users table (MySQL): " + e.getMessage());
        }
    }

    @Override
    public void save(User entity) {
        String sql = "INSERT INTO users(id, username, email, password) VALUES(?, ?, ?, ?)";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, entity.getId());
            pstmt.setString(2, entity.getUsername());
            pstmt.setString(3, entity.getEmail());
            pstmt.setString(4, entity.getPassword());
            pstmt.executeUpdate();
        } catch (SQLException e) {
            System.err.println("Error creating user (MySQL): " + e.getMessage());
        }
    }

    @Override
    public User findById(String id) {
        String sql = "SELECT * FROM users WHERE id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, id);
            ResultSet rs = pstmt.executeQuery();
            if (rs.next()) {
                return new User(
                    rs.getString("id"),
                    rs.getString("username"),
                    rs.getString("email"),
                    rs.getString("password")
                );
            }
        } catch (SQLException e) {
            System.err.println("Error finding user by id (MySQL): " + e.getMessage());
        }
        return null;
    }

    @Override
    public List<User> findAll() {
        List<User> users = new ArrayList<>();
        String sql = "SELECT * FROM users";
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                users.add(new User(
                    rs.getString("id"),
                    rs.getString("username"),
                    rs.getString("email"),
                    rs.getString("password")
```

```
                ));
            }
        } catch (SQLException e) {
            System.err.println("Error finding all users (MySQL): " + e.getMessage());
        }
        return users;
    }

    @Override
    public void update(User entity) {
        String sql = "UPDATE users SET username = ?, email = ?, password = ? WHERE id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, entity.getUsername());
            pstmt.setString(2, entity.getEmail());
            pstmt.setString(3, entity.getPassword());
            pstmt.setString(4, entity.getId());
            pstmt.executeUpdate();
        } catch (SQLException e) {
            System.err.println("Error updating user (MySQL): " + e.getMessage());
        }
    }

    @Override
    public void delete(String id) {
        String sql = "DELETE FROM users WHERE id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, id);
            pstmt.executeUpdate();
        } catch (SQLException e) {
            System.err.println("Error deleting user (MySQL): " + e.getMessage());
        }
    }

    protected Connection getConnection() throws SQLException { return DriverManager.getConnection(URL, USER_DB, PASS_DB); }
}
```

9.35 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/RepositoryFactory.java File Reference

```
import com.projectmanagement.lib.models.Project;
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.User;
```

Classes

- class com.projectmanagement.lib.storage.RepositoryFactory

Packages

- package com.projectmanagement.lib.storage

9.36 RepositoryFactory.java

Go to the documentation of this file.

```
package com.projectmanagement.lib.storage;

import com.projectmanagement.lib.models.Project;
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.User;

```

```
00011 public class RepositoryFactory {
00012
00016     private StorageConfig config;
00017
00022     public RepositoryFactory(StorageConfig config) {
00023         this.config = config;
00024     }
00025
00030     public IRepository<User> getUserRepository() {
00031         if (config.getActiveStorageType() == StorageType.BINARY_FILE) {
00032             return new BinaryUserRepository();
00033         } else if (config.getActiveStorageType() == StorageType.SQLITE) {
00034             return new SQLiteUserRepository();
00035         } else if (config.getActiveStorageType() == StorageType.MYSQL) {
00036             return new MySQLUserRepository();
00037         }
00038         throw new IllegalStateException("Unsupported storage type for Users");
00039     }
00040
00045     public IRepository<Project> getProjectRepository() {
00046         if (config.getActiveStorageType() == StorageType.BINARY_FILE) {
00047             return new BinaryProjectRepository();
00048         } else if (config.getActiveStorageType() == StorageType.SQLITE) {
00049             return new SQLiteProjectRepository();
00050         } else if (config.getActiveStorageType() == StorageType.MYSQL) {
00051             return new MySQLProjectRepository();
00052         }
00053         throw new IllegalStateException("Unsupported storage type for Projects");
00054     }
00055
00060     public IRepository<Task> getTaskRepository() {
00061         if (config.getActiveStorageType() == StorageType.BINARY_FILE) {
00062             return new BinaryTaskRepository();
00063         } else if (config.getActiveStorageType() == StorageType.SQLITE) {
00064             return new SQLiteTaskRepository();
00065         } else if (config.getActiveStorageType() == StorageType.MYSQL) {
00066             return new MySQLTaskRepository();
00067         }
00068         throw new IllegalStateException("Unsupported storage type for Tasks");
00069     }
00070 }
```

9.37 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/SQLiteProjectRepository.java File Reference

```
import com.projectmanagement.lib.models.Project;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;
```

Classes

- class com.projectmanagement.lib.storage.SQLiteProjectRepository

Packages

- package com.projectmanagement.lib.storage

9.38 SQLiteProjectRepository.java

Go to the documentation of this file.

```
00001 package com.projectmanagement.lib.storage;
00002
00003 import com.projectmanagement.lib.models.Project;
00004 import java.sql.*;
00005 import java.util.ArrayList;
00006 import java.util.List;
00007
00012 public class SQLiteProjectRepository implements IRepository<Project> {
00013
00017     private static final String URL = "jdbc:sqlite:projectmanager.db";
00018
00022     public SQLiteProjectRepository() {
00023         createTableIfNotExists();
00024     }
00025
00029     private void createTableIfNotExists() {
00030         String sql = "CREATE TABLE IF NOT EXISTS projects ("
00031                    + "id TEXT PRIMARY KEY,"
00032                    + "name TEXT NOT NULL,"
00033                    + "description TEXT"
00034                    + ");";
00035         String mappingSql = "CREATE TABLE IF NOT EXISTS project_tasks ("
00036                    + "project_id TEXT,"
00037                    + "task_id TEXT,"
00038                    + "PRIMARY KEY (project_id, task_id)"
00039                    + ");";
00040         try (Connection conn = getConnection();
00041              Statement stmt = conn.createStatement()) {
00042             stmt.execute(sql);
00043             stmt.execute(mappingSql);
00044         } catch (SQLException e) {
00045             System.err.println("Error creating projects table: " + e.getMessage());
00046         }
00047     }
00048
00053     @Override
00054     public void save(Project entity) {
00055         String sql = "INSERT INTO projects(id, name, description) VALUES(?, ?, ?)";
00056         try (Connection conn = getConnection();
00057              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00058             pstmt.setString(1, entity.getId());
00059             pstmt.setString(2, entity.getName());
00060             pstmt.setString(3, entity.getDescription());
00061             pstmt.executeUpdate();
00062
00063             saveProjectTasks(conn, entity);
00064         } catch (SQLException e) {
00065             System.err.println("Error creating project: " + e.getMessage());
00066         }
00067     }
00068
00069     private void saveProjectTasks(Connection conn, Project entity) throws SQLException {
00070         String delSql = "DELETE FROM project_tasks WHERE project_id = ?";
00071         try (PreparedStatement delStmt = conn.prepareStatement(delSql)) {
00072             delStmt.setString(1, entity.getId());
00073             delStmt.executeUpdate();
00074         }
00075
00076         if (entity.getTasks() != null && !entity.getTasks().isEmpty()) {
00077             String insSql = "INSERT INTO project_tasks (project_id, task_id) VALUES (?, ?)";
00078             try (PreparedStatement insStmt = conn.prepareStatement(insSql)) {
00079                 for (com.projectmanagement.lib.models.Task task : entity.getTasks()) {
00080                     insStmt.setString(1, entity.getId());
00081                     insStmt.setString(2, task.getId());
00082                     insStmt.executeUpdate();
00083                 }
00084             }
00085         }
00086     }
00087
00093     @Override
00094     public Project findById(String id) {
00095         String sql = "SELECT * FROM projects WHERE id = ?";
00096         try (Connection conn = getConnection();
00097              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00098             pstmt.setString(1, id);
00099             ResultSet rs = pstmt.executeQuery();
00100             if (rs.next()) {
00101                 Project project = new Project(
00102                     rs.getString("id"),
00103                     rs.getString("name"),
00104                     rs.getString("description")
```

```
                );
                loadProjectTasks(conn, project);
                return project;
            }
        } catch (SQLException e) {
            System.err.println("Error finding project by id: " + e.getMessage());
        }
        return null;
    }

    private void loadProjectTasks(Connection conn, Project project) throws SQLException {
        String taskSql = "SELECT task_id FROM project_tasks WHERE project_id = ?";
        try (PreparedStatement tpstmt = conn.prepareStatement(taskSql)) {
            tpstmt.setString(1, project.getId());
            ResultSet trs = tpstmt.executeQuery();
            SQLiteTaskRepository taskRepo = new SQLiteTaskRepository();
            while (trs.next()) {
                com.projectmanagement.lib.models.Task task =
taskRepo.findById(trs.getString("task_id"));
                if (task != null) {
                    project.addTask(task);
                }
            }
        }
    }

    @Override
    public List<Project> findAll() {
        List<Project> projects = new ArrayList<>();
        String sql = "SELECT * FROM projects";
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                Project project = new Project(
                    rs.getString("id"),
                    rs.getString("name"),
                    rs.getString("description")
                );
                loadProjectTasks(conn, project);
                projects.add(project);
            }
        } catch (SQLException e) {
            System.err.println("Error finding all projects: " + e.getMessage());
        }
        return projects;
    }

    @Override
    public void update(Project entity) {
        String sql = "UPDATE projects SET name = ?, description = ? WHERE id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, entity.getName());
            pstmt.setString(2, entity.getDescription());
            pstmt.setString(3, entity.getId());
            pstmt.executeUpdate();

            saveProjectTasks(conn, entity);
        } catch (SQLException e) {
            System.err.println("Error updating project: " + e.getMessage());
        }
    }

    @Override
    public void delete(String id) {
        String sql = "DELETE FROM projects WHERE id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, id);
            pstmt.executeUpdate();

            String delSql = "DELETE FROM project_tasks WHERE project_id = ?";
            try (PreparedStatement delStmt = conn.prepareStatement(delSql)) {
                delStmt.setString(1, id);
                delStmt.executeUpdate();
            }
        } catch (SQLException e) {
            System.err.println("Error deleting project: " + e.getMessage());
        }
    }

    protected Connection getConnection() throws SQLException { return
DriverManager.getConnection(URL); }
}
```

9.39 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/SQLiteTaskRepository.java File Reference

```
import com.projectmanagement.lib.models.Task;
import com.projectmanagement.lib.models.TaskStatus;
import com.projectmanagement.lib.models.User;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;
```

Classes

- class com.projectmanagement.lib.storage.SQLiteTaskRepository

Packages

- package com.projectmanagement.lib.storage

9.40 SQLiteTaskRepository.java

Go to the documentation of this file.

```
00001 package com.projectmanagement.lib.storage;
00002
00003 import com.projectmanagement.lib.models.Task;
00004 import com.projectmanagement.lib.models.TaskStatus;
00005 import com.projectmanagement.lib.models.User;
00006 import java.sql.*;
00007 import java.util.ArrayList;
00008 import java.util.List;
00009
00014 public class SQLiteTaskRepository implements IRepository<Task> {
00015
00019     private static final String URL = "jdbc:sqlite:projectmanager.db";
00020
00024     public SQLiteTaskRepository() {
00025         createTableIfNotExists();
00026     }
00027
00031     private void createTableIfNotExists() {
00032         String sql = "CREATE TABLE IF NOT EXISTS tasks ("
00033                    + "id TEXT PRIMARY KEY,"
00034                    + "title TEXT NOT NULL,"
00035                    + "description TEXT,"
00036                    + "status TEXT NOT NULL,"
00037                    + "assigned_user_id TEXT"
00038                    + ");";
00039         try (Connection conn = getConnection();
00040              Statement stmt = conn.createStatement()) {
00041             stmt.execute(sql);
00042         } catch (SQLException e) {
00043             System.err.println("Error creating tasks table: " + e.getMessage());
00044         }
00045     }
00046
00051     @Override
00052     public void save(Task entity) {
00053         String sql = "INSERT INTO tasks(id, title, description, status, assigned_user_id) VALUES(?, ?,
      ?, ?, ?)";
00054         try (Connection conn = getConnection();
00055              PreparedStatement pstmt = conn.prepareStatement(sql)) {
00056             pstmt.setString(1, entity.getId());
00057             pstmt.setString(2, entity.getTitle());
00058             pstmt.setString(3, entity.getDescription());
00059             pstmt.setString(4, entity.getStatus().name());
00060             pstmt.setString(5, entity.getAssignedUser() != null ? entity.getAssignedUser().getId() :
      null);
```

```
            pstmt.executeUpdate();
        } catch (SQLException e) {
            System.err.println("Error creating task: " + e.getMessage());
        }
    }

    @Override
    public Task findById(String id) {
        String sql = "SELECT * FROM tasks WHERE id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, id);
            ResultSet rs = pstmt.executeQuery();
            if (rs.next()) {
                Task task = new Task(
                    rs.getString("id"),
                    rs.getString("title"),
                    rs.getString("description")
                );
                task.setStatus(TaskStatus.valueOf(rs.getString("status")));
                String userId = rs.getString("assigned_user_id");
                if (userId != null) {
                    User assignee = new SQLiteUserRepository().findById(userId);
                    task.setAssignedUser(assignee);
                }
                return task;
            }
        } catch (SQLException e) {
            System.err.println("Error finding task by id: " + e.getMessage());
        }
        return null;
    }

    @Override
    public List<Task> findAll() {
        List<Task> tasks = new ArrayList<>();
        String sql = "SELECT * FROM tasks";
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            SQLiteUserRepository userRepo = new SQLiteUserRepository();
            while (rs.next()) {
                Task task = new Task(
                    rs.getString("id"),
                    rs.getString("title"),
                    rs.getString("description")
                );
                task.setStatus(TaskStatus.valueOf(rs.getString("status")));
                String userId = rs.getString("assigned_user_id");
                if (userId != null) {
                    task.setAssignedUser(userRepo.findById(userId));
                }
                tasks.add(task);
            }
        } catch (SQLException e) {
            System.err.println("Error finding all tasks: " + e.getMessage());
        }
        return tasks;
    }

    @Override
    public void update(Task entity) {
        String sql = "UPDATE tasks SET title = ?, description = ?, status = ?, assigned_user_id = ?
WHERE id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, entity.getTitle());
            pstmt.setString(2, entity.getDescription());
            pstmt.setString(3, entity.getStatus().name());
            pstmt.setString(4, entity.getAssignedUser() != null ? entity.getAssignedUser().getId() :
null);
            pstmt.setString(5, entity.getId());
            pstmt.executeUpdate();
        } catch (SQLException e) {
            System.err.println("Error updating task: " + e.getMessage());
        }
    }

    @Override
    public void delete(String id) {
        String sql = "DELETE FROM tasks WHERE id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, id);
            pstmt.executeUpdate();
        } catch (SQLException e) {
            System.err.println("Error deleting task: " + e.getMessage());
```

```
        }
    }

    protected Connection getConnection() throws SQLException { return DriverManager.getConnection(URL); }
}
```

9.41 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/SQLiteUserRepository.java File Reference

```
import com.projectmanagement.lib.models.User;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;
```

Classes

- class com.projectmanagement.lib.storage.SQLiteUserRepository

Packages

- package com.projectmanagement.lib.storage

9.42 SQLiteUserRepository.java

Go to the documentation of this file.

```
package com.projectmanagement.lib.storage;

import com.projectmanagement.lib.models.User;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class SQLiteUserRepository implements IRepository<User> {

    private static final String URL = "jdbc:sqlite:projectmanager.db";

    public SQLiteUserRepository() {
        createTableIfNotExists();
    }

    private void createTableIfNotExists() {
        String sql = "CREATE TABLE IF NOT EXISTS users ("
                   + "id TEXT PRIMARY KEY,"
                   + "username TEXT NOT NULL,"
                   + "email TEXT NOT NULL,"
                   + "password TEXT NOT NULL"
                   + ");";
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement()) {
            stmt.execute(sql);
        } catch (SQLException e) {
            System.err.println("Error creating users table: " + e.getMessage());
        }
    }

    @Override
    public void save(User entity) {
        String sql = "INSERT INTO users(id, username, email, password) VALUES(?, ?, ?, ?)";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, entity.getId());
            pstmt.setString(2, entity.getUsername());
```

```
            pstmt.setString(3, entity.getEmail());
            pstmt.setString(4, entity.getPassword());
            pstmt.executeUpdate();
        } catch (SQLException e) {
            System.err.println("Error creating user: " + e.getMessage());
        }
    }

    @Override
    public User findById(String id) {
        String sql = "SELECT * FROM users WHERE id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, id);
            ResultSet rs = pstmt.executeQuery();
            if (rs.next()) {
                return new User(
                    rs.getString("id"),
                    rs.getString("username"),
                    rs.getString("email"),
                    rs.getString("password")
                );
            }
        } catch (SQLException e) {
            System.err.println("Error finding user by id: " + e.getMessage());
        }
        return null;
    }

    @Override
    public List<User> findAll() {
        List<User> users = new ArrayList<>();
        String sql = "SELECT * FROM users";
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                users.add(new User(
                    rs.getString("id"),
                    rs.getString("username"),
                    rs.getString("email"),
                    rs.getString("password")
                ));
            }
        } catch (SQLException e) {
            System.err.println("Error finding all users: " + e.getMessage());
        }
        return users;
    }

    @Override
    public void update(User entity) {
        String sql = "UPDATE users SET username = ?, email = ?, password = ? WHERE id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, entity.getUsername());
            pstmt.setString(2, entity.getEmail());
            pstmt.setString(3, entity.getPassword());
            pstmt.setString(4, entity.getId());
            pstmt.executeUpdate();
        } catch (SQLException e) {
            System.err.println("Error updating user: " + e.getMessage());
        }
    }

    @Override
    public void delete(String id) {
        String sql = "DELETE FROM users WHERE id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, id);
            pstmt.executeUpdate();
        } catch (SQLException e) {
            System.err.println("Error deleting user: " + e.getMessage());
        }
    }

    protected Connection getConnection() throws SQLException { return
DriverManager.getConnection(URL); }
}
```

9.43 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/StorageConfig.java File Reference

Classes

- class com.projectmanagement.lib.storage.StorageConfig

Packages

- package com.projectmanagement.lib.storage

9.44 StorageConfig.java

Go to the documentation of this file.

```
00001 package com.projectmanagement.lib.storage;
00002
00007 public class StorageConfig {
00008
00012     private StorageType activeStorageType;
00013
00018     public StorageConfig(StorageType defaultStorage) {
00019         this.activeStorageType = defaultStorage;
00020     }
00021
00026     public StorageType getActiveStorageType() {
00027         return activeStorageType;
00028     }
00029
00034     public void setActiveStorageType(StorageType activeStorageType) {
00035         this.activeStorageType = activeStorageType;
00036     }
00037 }
```

9.45 projectmanager-app/src/main/java/com/projectmanagement/lib/storage/StorageType.java File Reference

Classes

- enum com.projectmanagement.lib.storage.StorageType

Packages

- package com.projectmanagement.lib.storage

9.46 StorageType.java

Go to the documentation of this file.

```
00001 package com.projectmanagement.lib.storage;
00002
00006 public enum StorageType {
00010     BINARY_FILE,
00011
00015     SQLITE,
00016
00020     MYSQL
00021 }
```